

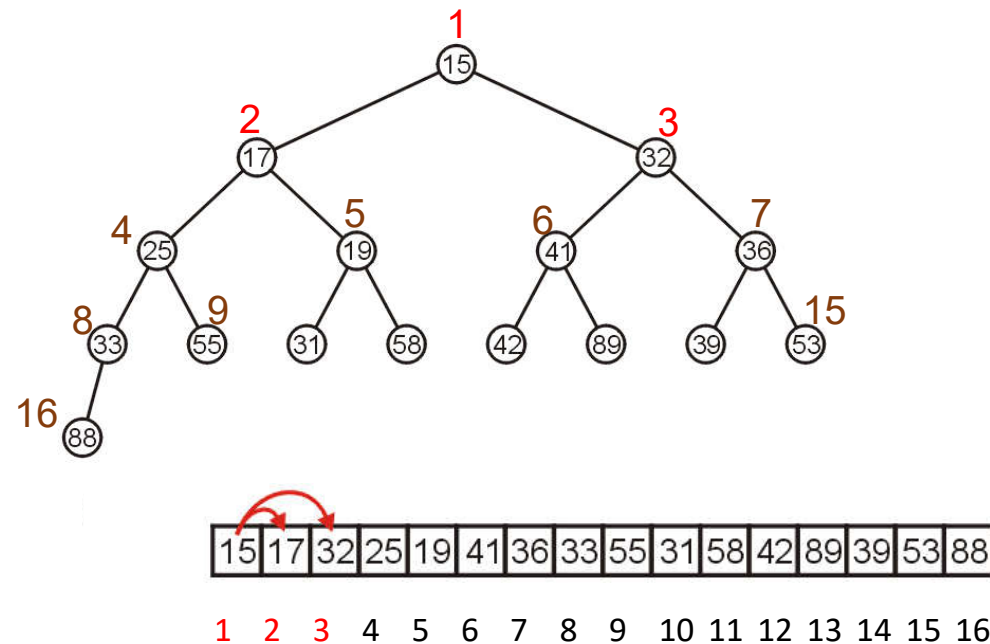
CSE 105: Data Structures and Algorithms-I (Part 2)

Instructor
Dr Md Monirul Islam

Heap, Heapsort and Priority Queue

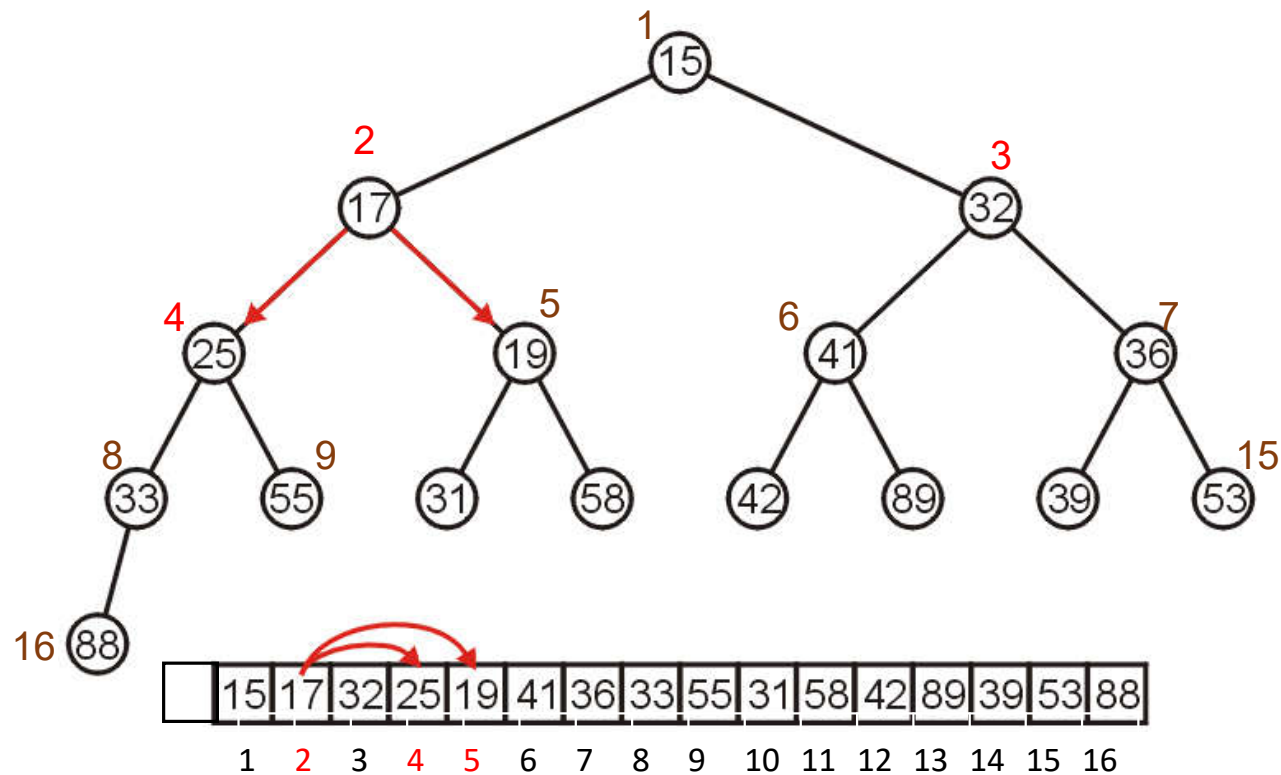
Array Implementation

The children of 15 are 17 and 32:



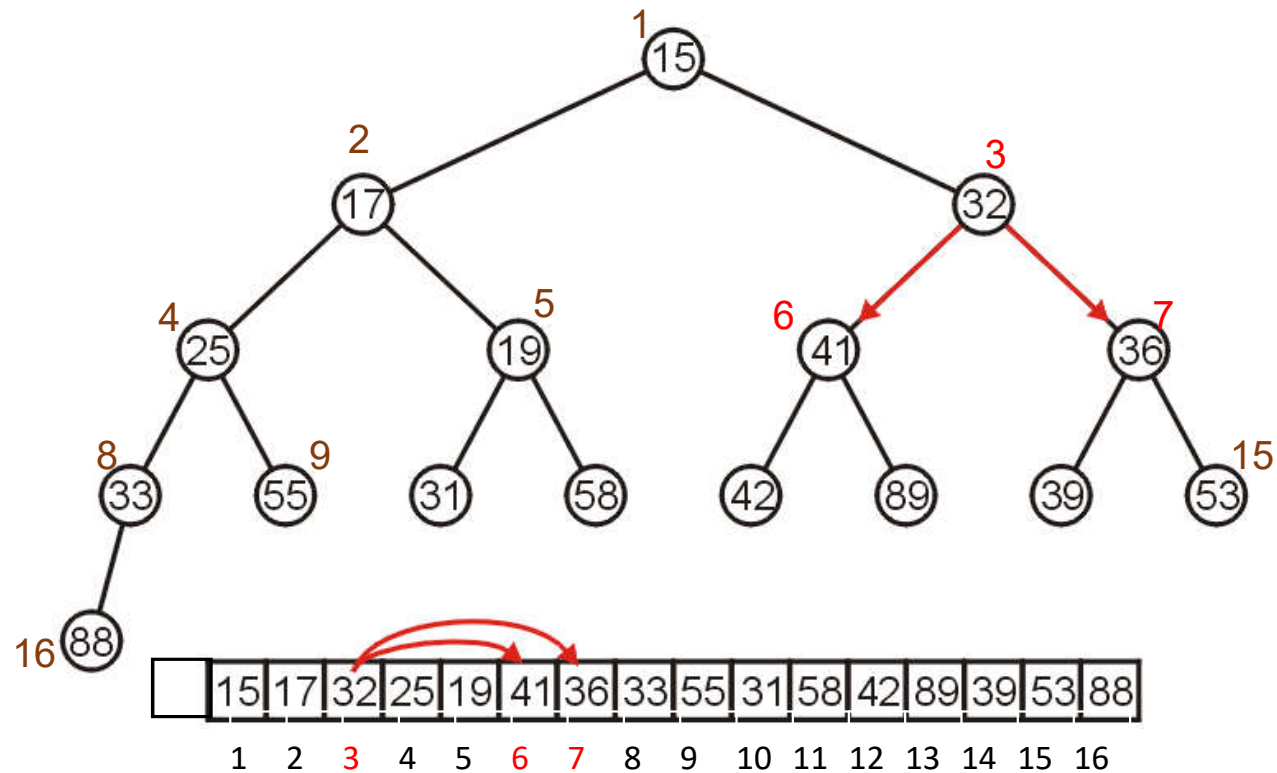
Array Implementation

The children of 17 are 25 and 19:



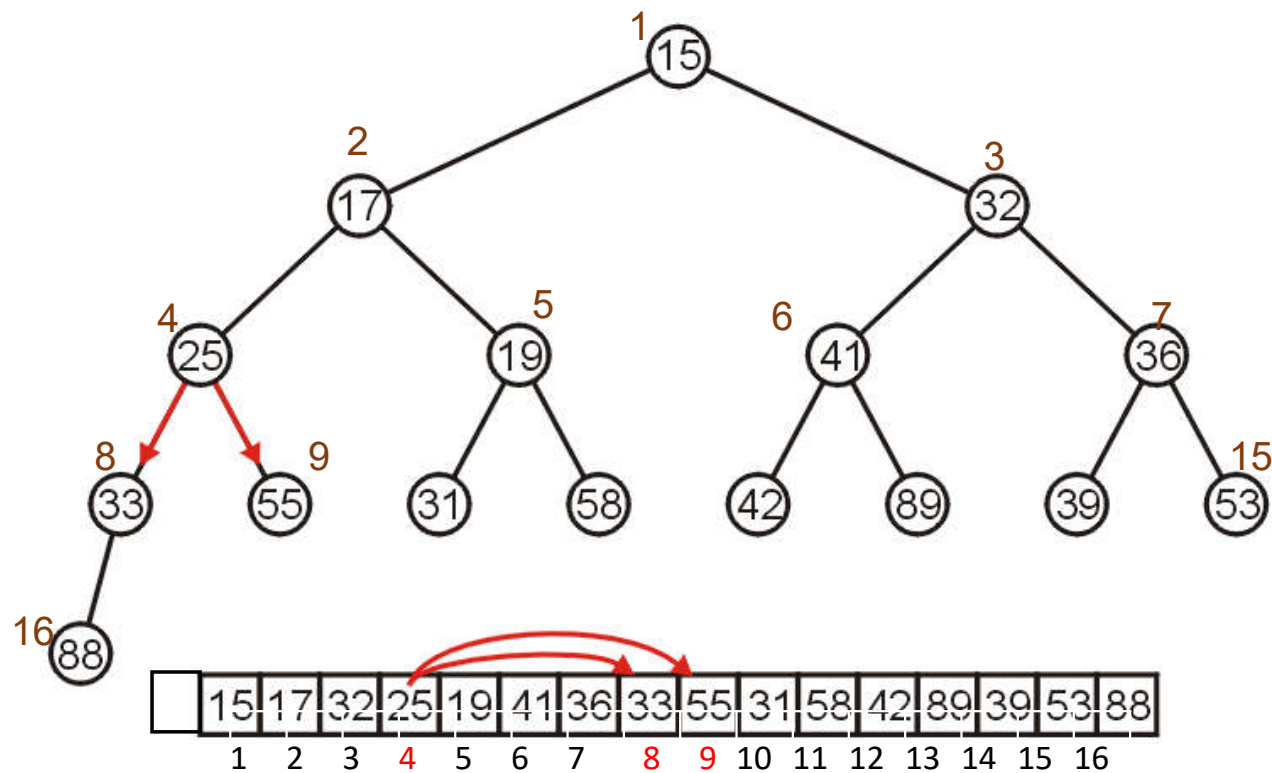
Array Implementation

The children of 32 are 41 and 36:

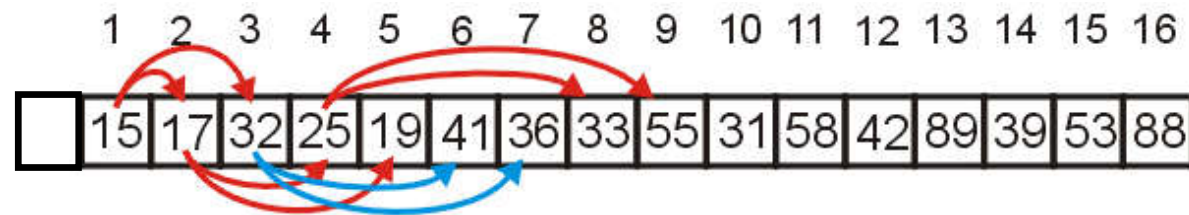
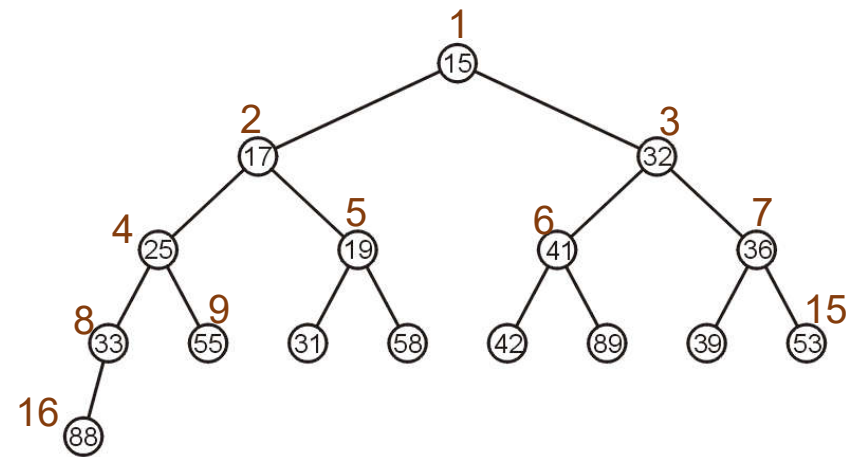


Array Implementation

The children of 25 are 33 and 55:



Array Implementation



Min-heap property to be maintained:

$$Array[parent(i)] \leq Array[i]$$

Array Implementation: Push

Let heap-as-array has n entries

then the next empty node in the corresponding complete tree is at location $posn = n + 1$

While percolating up,

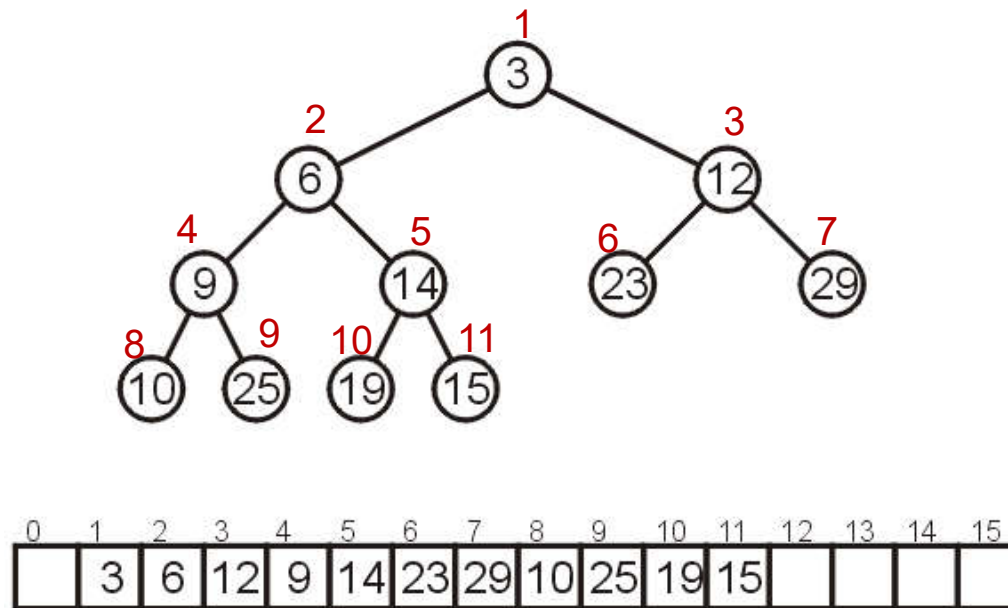
compare the item at location $posn$ with the item at $posn/2$

if they are out of order

- swap them, then set $posn = posn/2$ and repeat

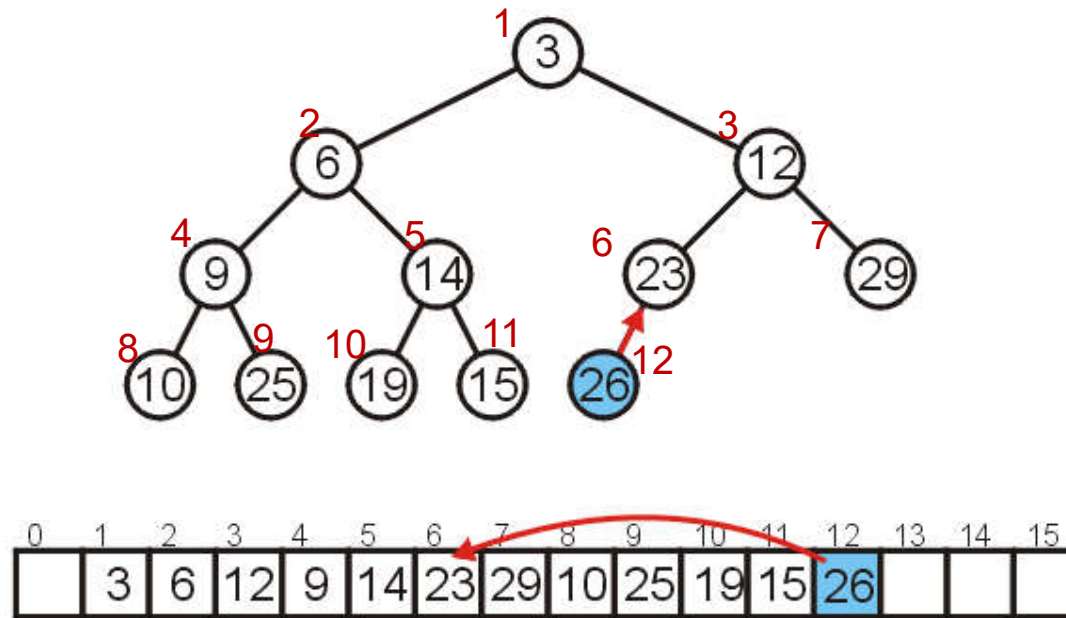
Array Implementation: Push

Consider the following heap, both as a tree and in its array representation



Array Implementation: Push

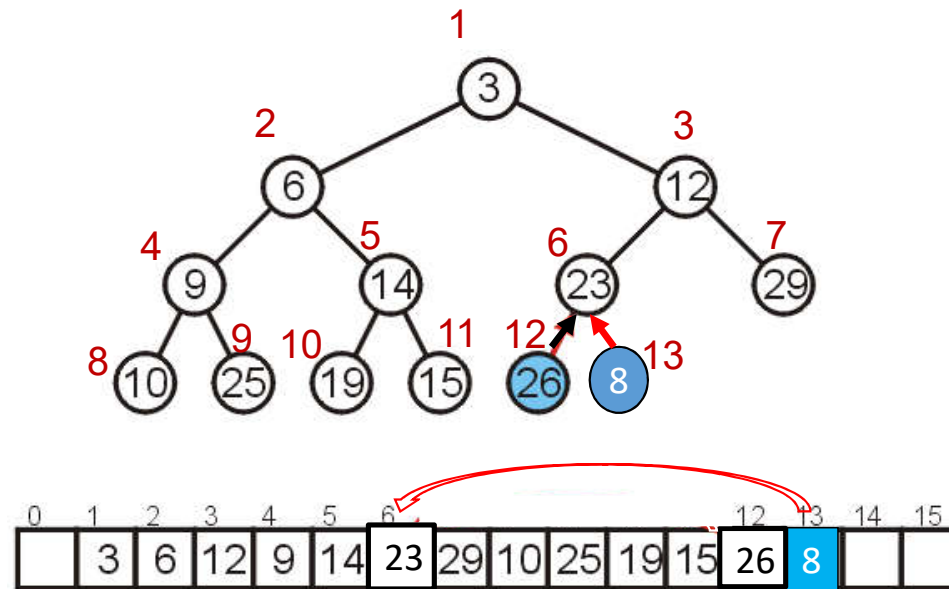
Inserting 26 requires no changes



Array Implementation: Push

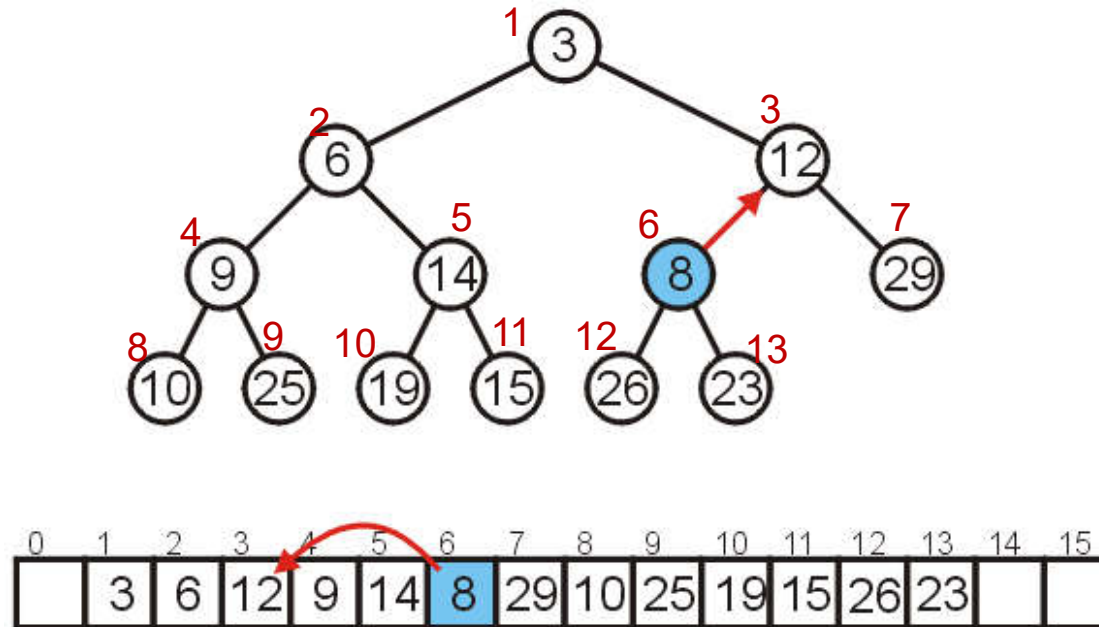
Inserting 8 requires a few percolations:

- Swap 8 and 23



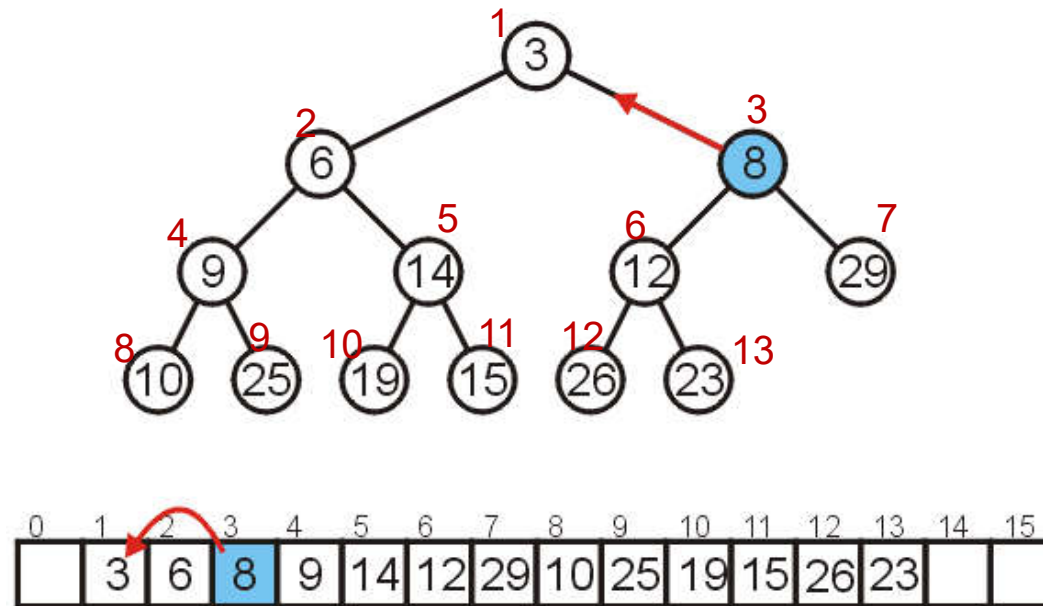
Array Implementation: Push

Swap 8 and 12



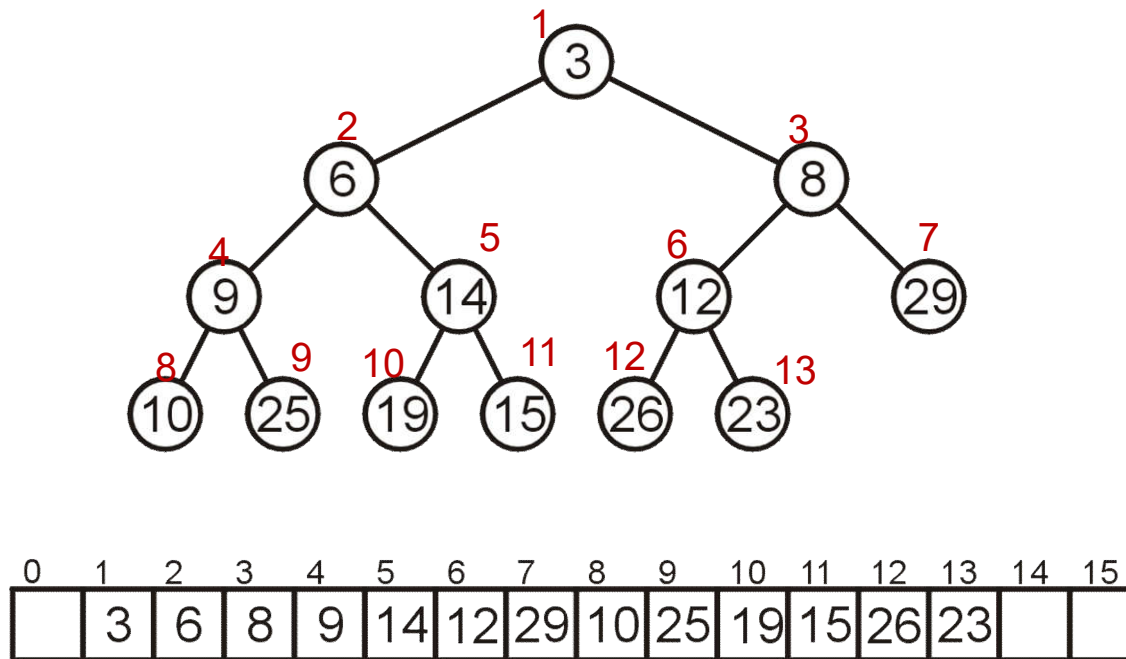
Array Implementation: Push

At this point, it is **greater than** its **parent**, so we are **finished**



Array Implementation: Pop

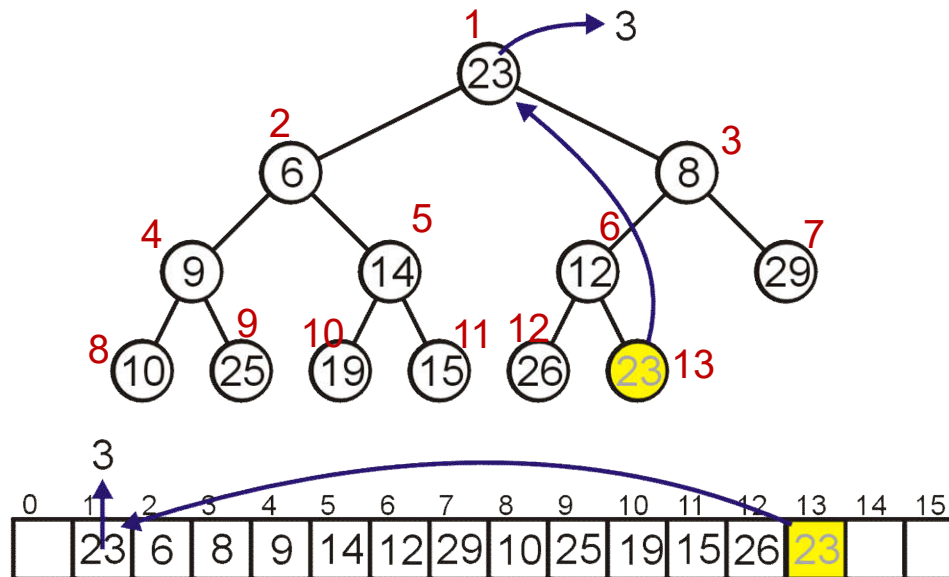
As before, popping the top has us copy the last entry to the top



Array Implementation: Pop

Instead, consider this strategy:

- Copy the last object, 23, to the root

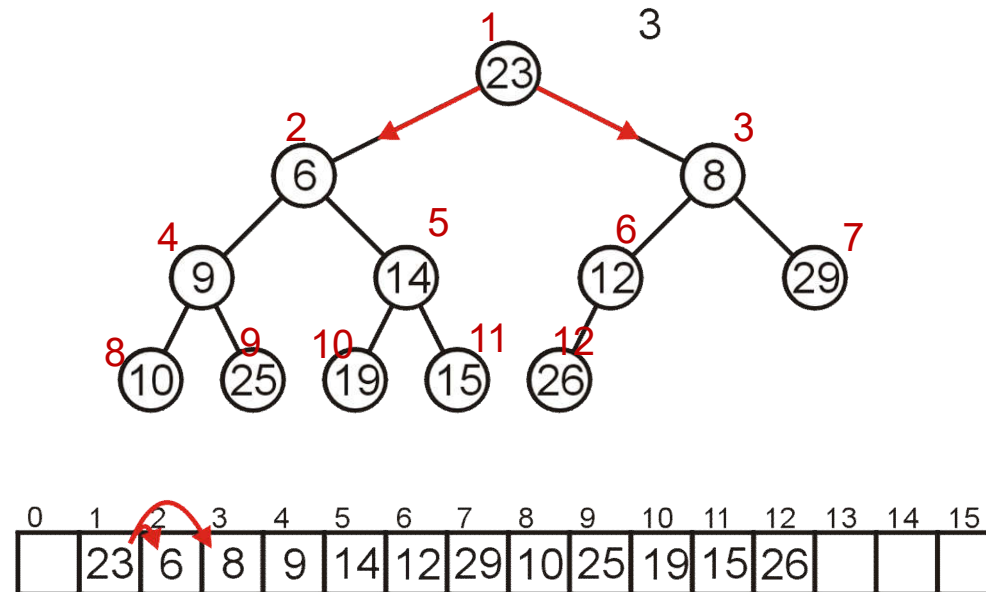


Array Implementation: Pop

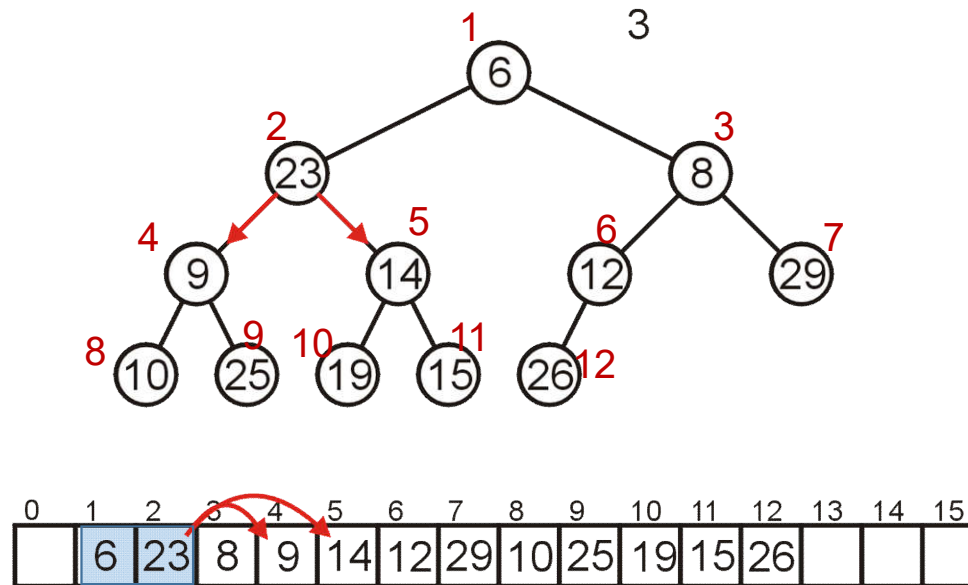
Now percolate down

Compare Node 1 with its children: Nodes 2 and 3

- Swap 23 and 6



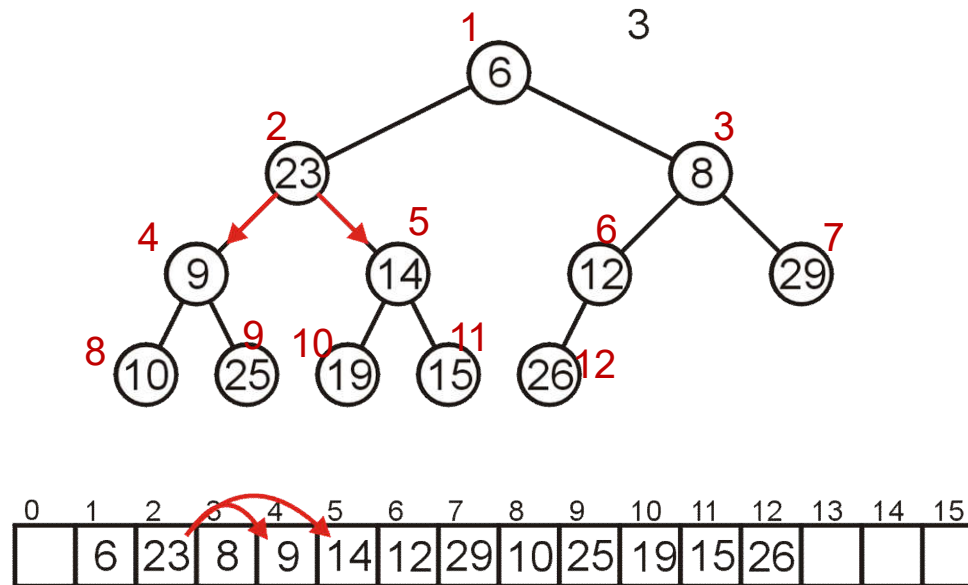
Array Implementation: Pop



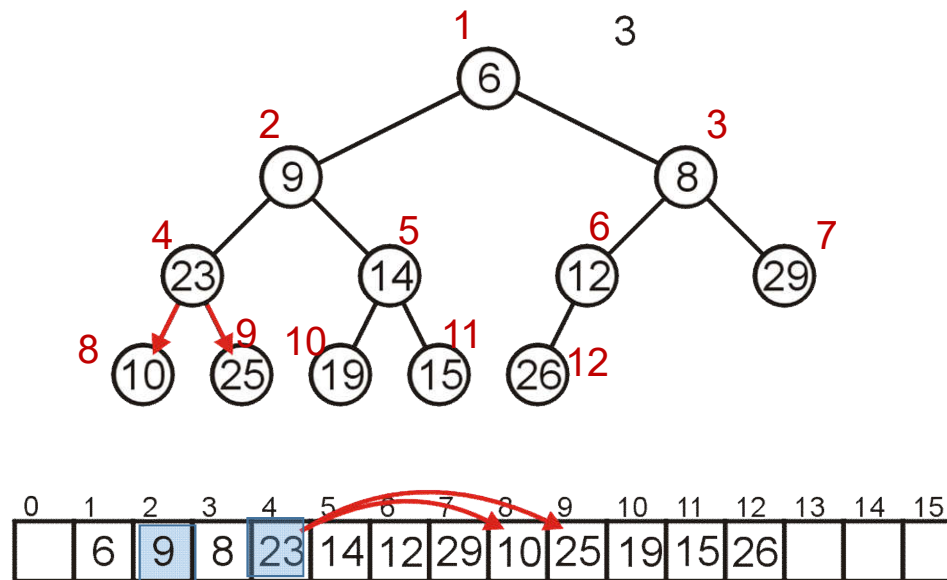
Array Implementation: Pop

Compare Node 2 with its children: Nodes 4 and 5

- Swap 23 and 9



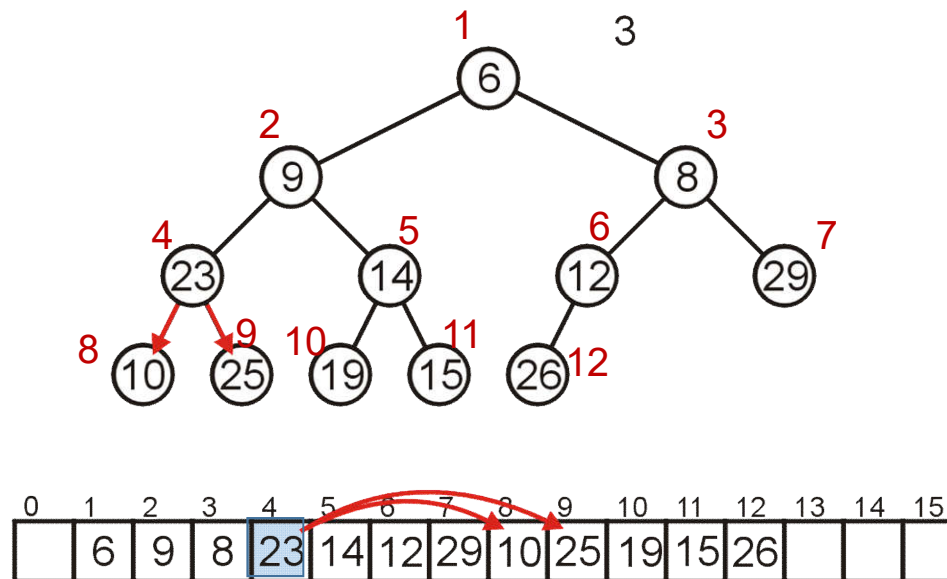
Array Implementation: Pop



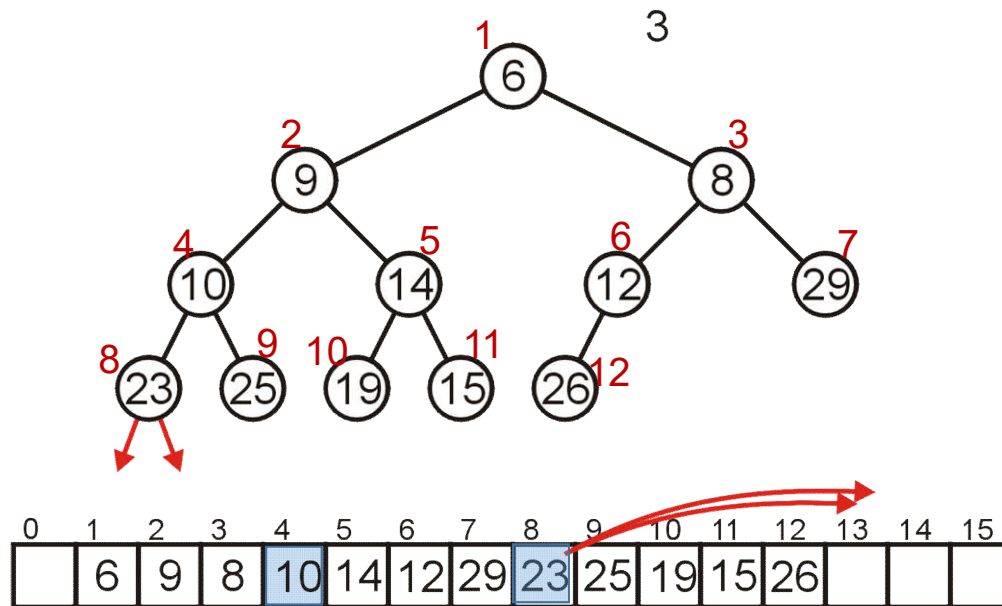
Array Implementation: Pop

Compare Node 4 with its children: Nodes 8 and 9

- Swap 23 and 10



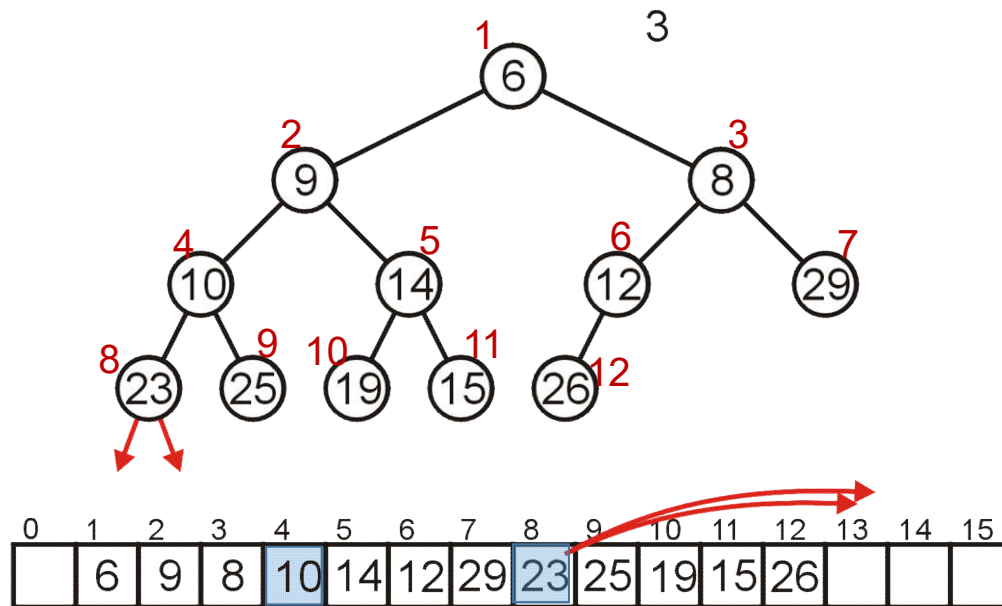
Array Implementation: Pop



Array Implementation: Pop

The children of Node 8 are beyond the end of the array:

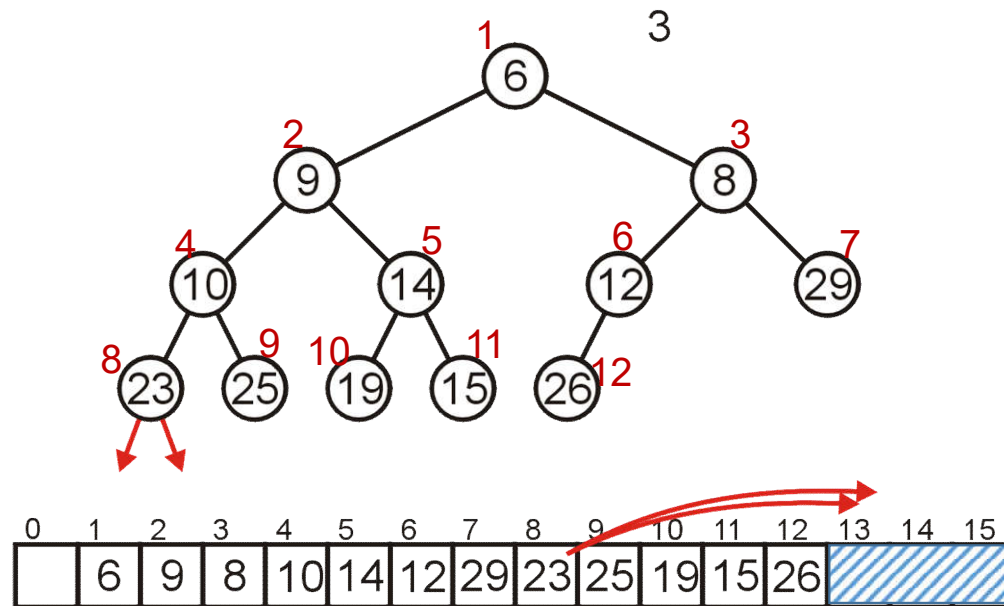
- Stop



Array Implementation: Pop

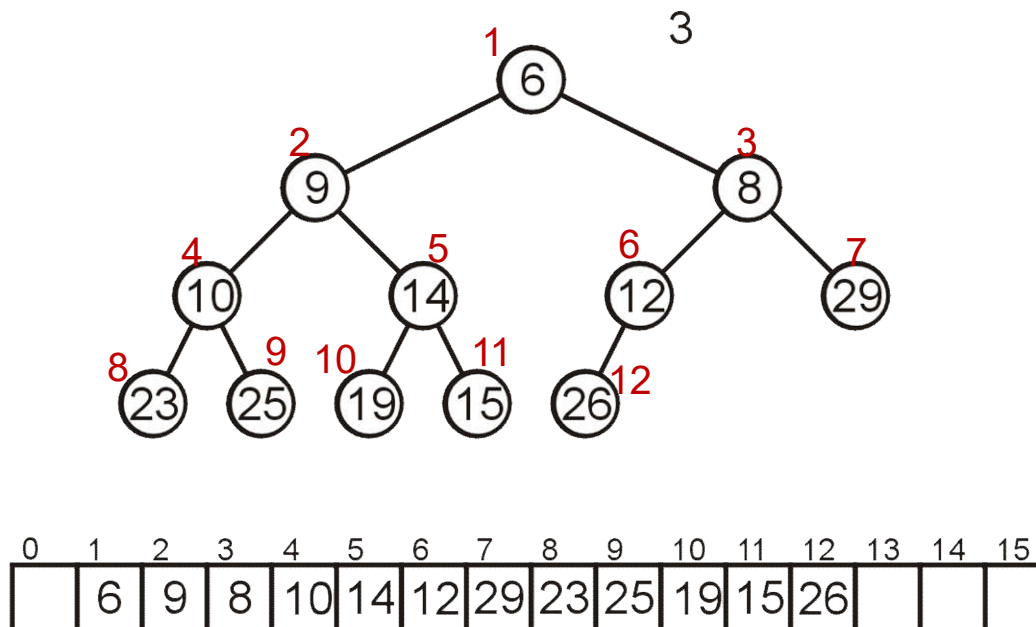
The children of Node 8 are beyond the end of the array:

- Stop



Array Implementation: Pop

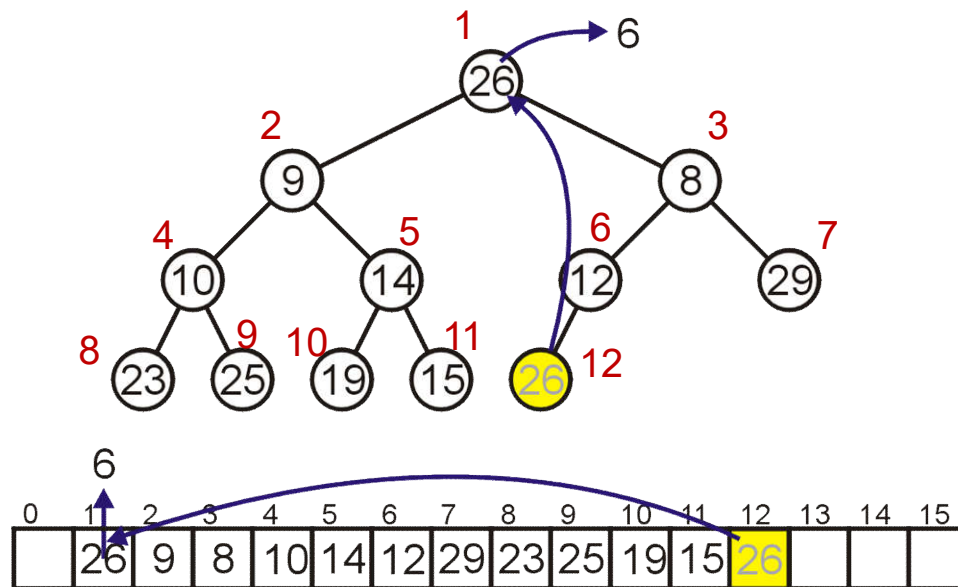
The result is a binary min-heap



Array Implementation: Pop

Dequeuing the minimum again:

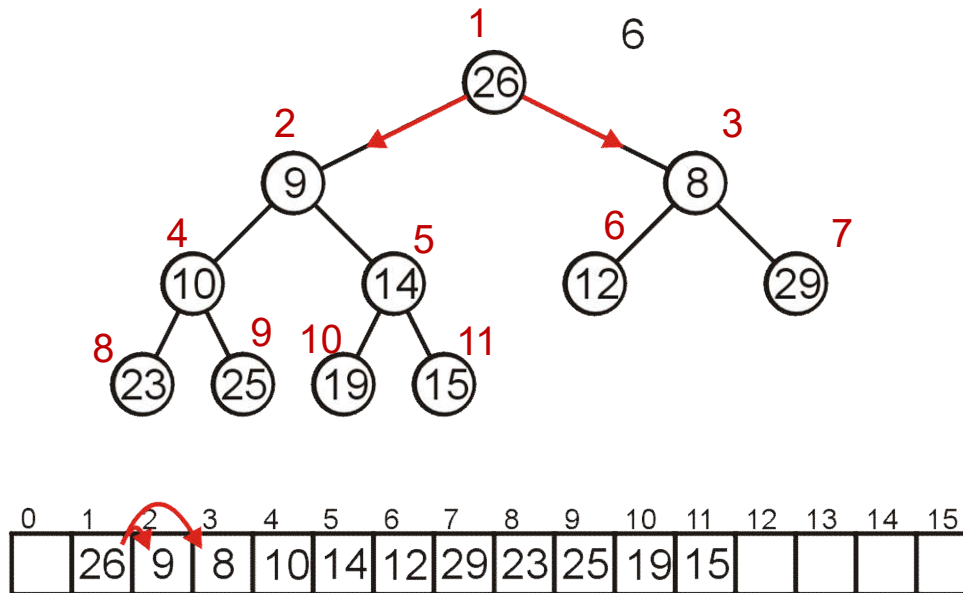
- Copy 26 to the root



Array Implementation: Pop

Compare Node 1 with its children: Nodes 2 and 3

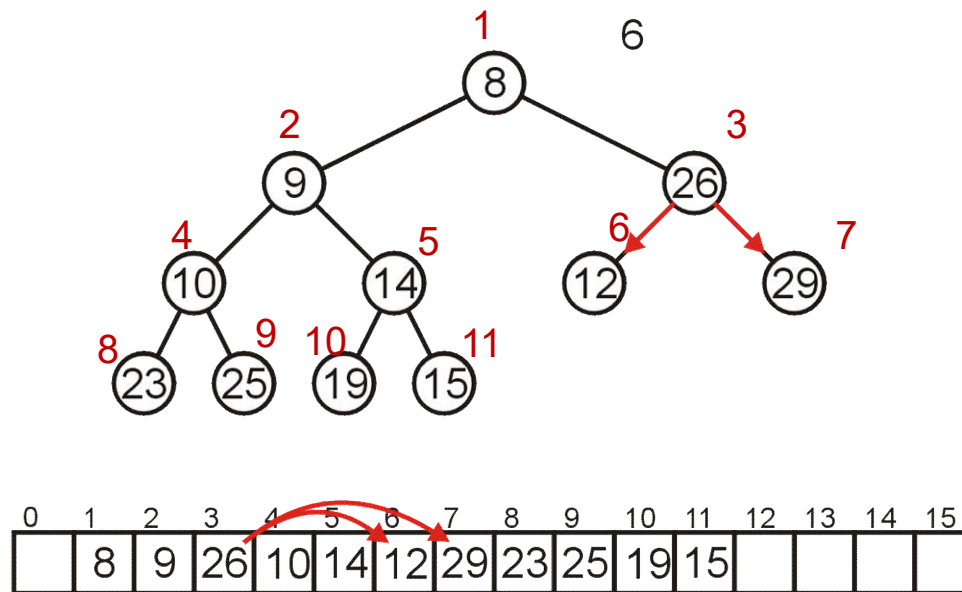
- Swap 26 and 8



Array Implementation: Pop

Compare Node 3 with its children: Nodes 6 and 7

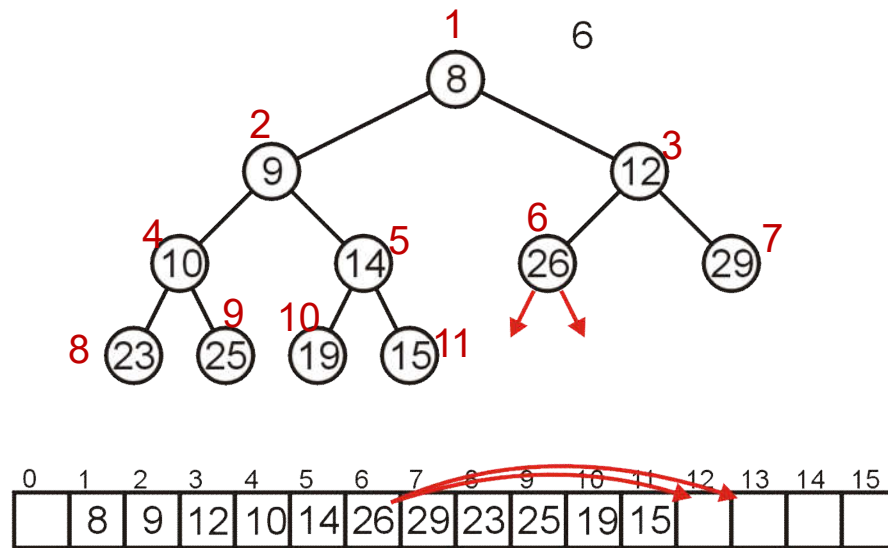
- Swap 26 and 12



Array Implementation: Pop

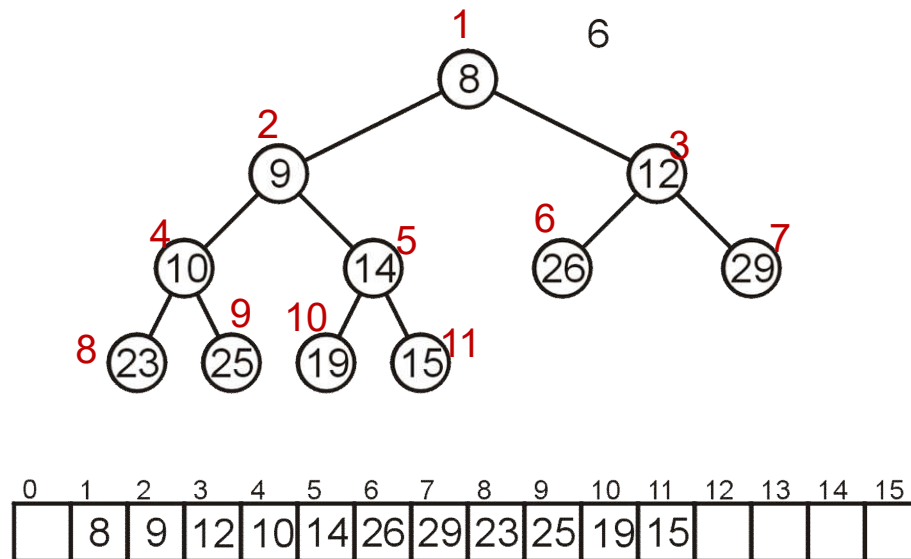
The children of Node 6, Nodes 12 and 13 are unoccupied

- Currently, count == 11



Array Implementation: Pop

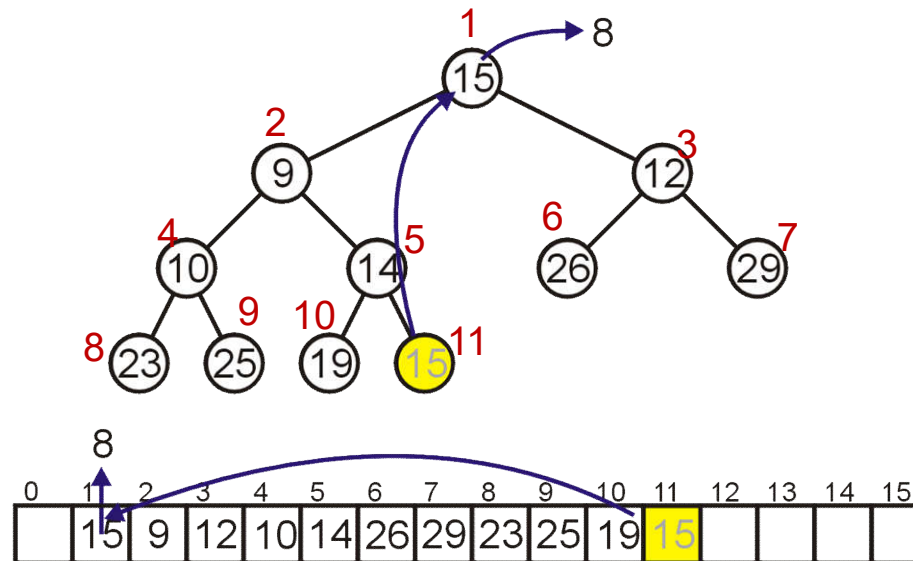
The result is a min-heap



Array Implementation: Pop

Dequeuing the minimum a third time:

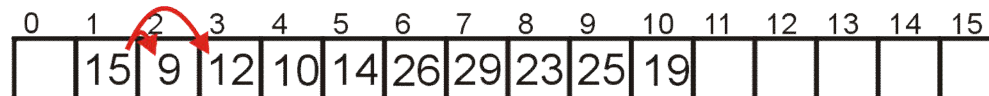
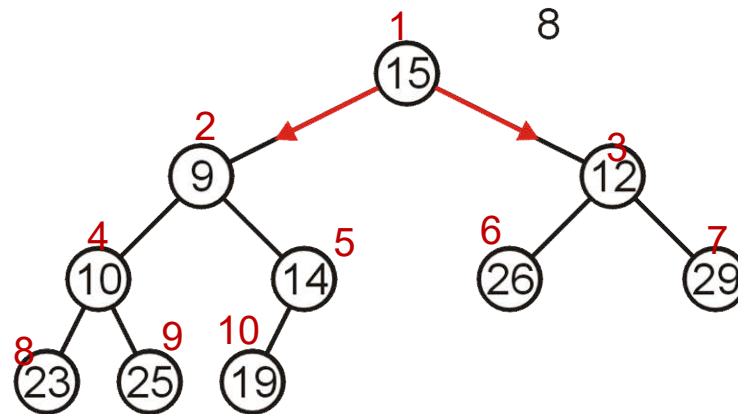
- Copy 15 to the root



Array Implementation: Pop

Compare Node 1 with its children: Nodes 2 and 3

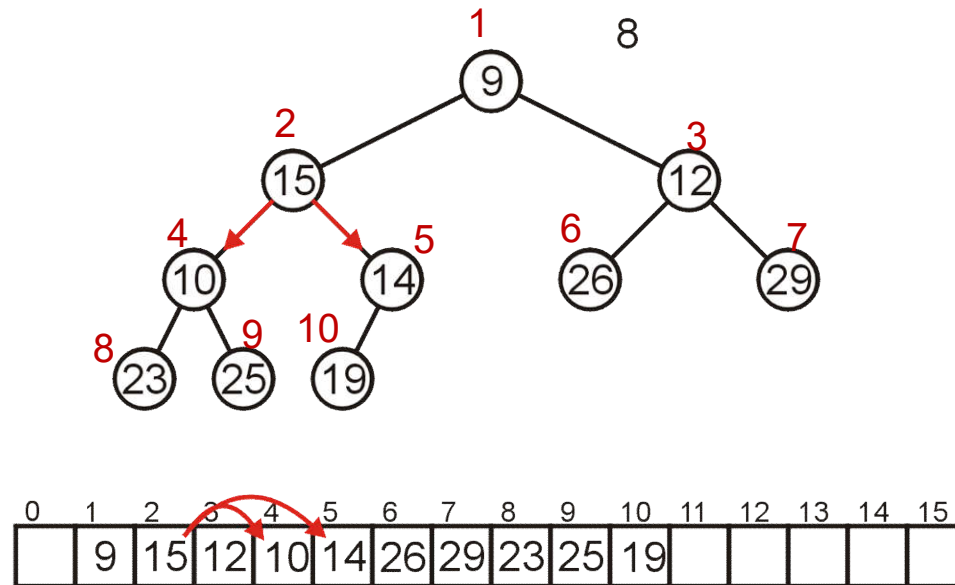
- Swap 15 and 9



Array Implementation: Pop

Compare Node 2 with its children: Nodes 4 and 5

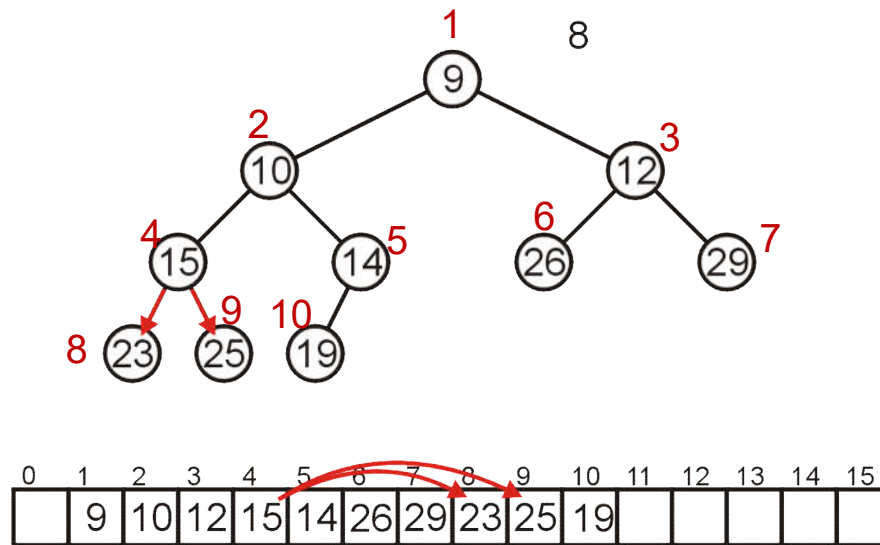
- Swap 15 and 10



Array Implementation: Pop

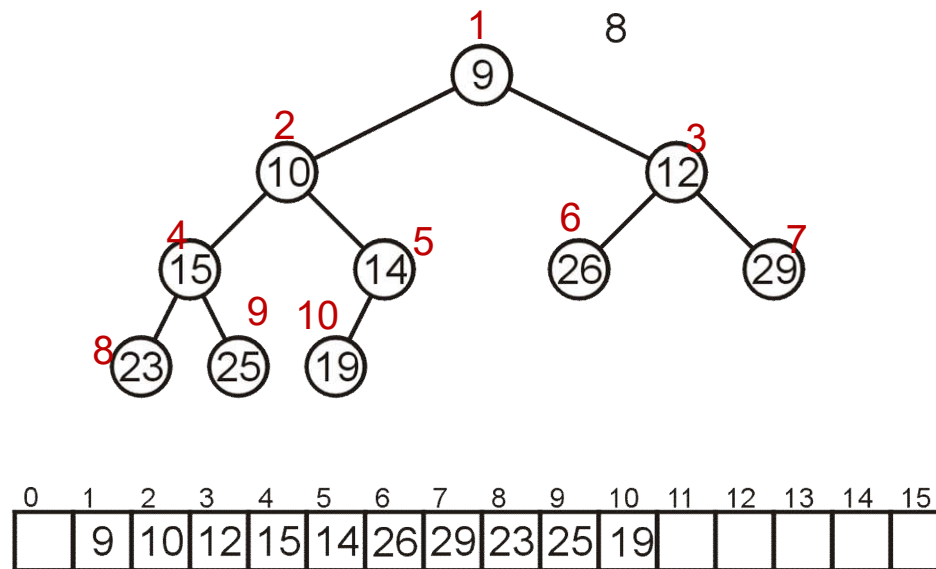
Compare Node 4 with its children: Nodes 8 and 9

- $15 < 23$ and $15 < 25$, so **stop**



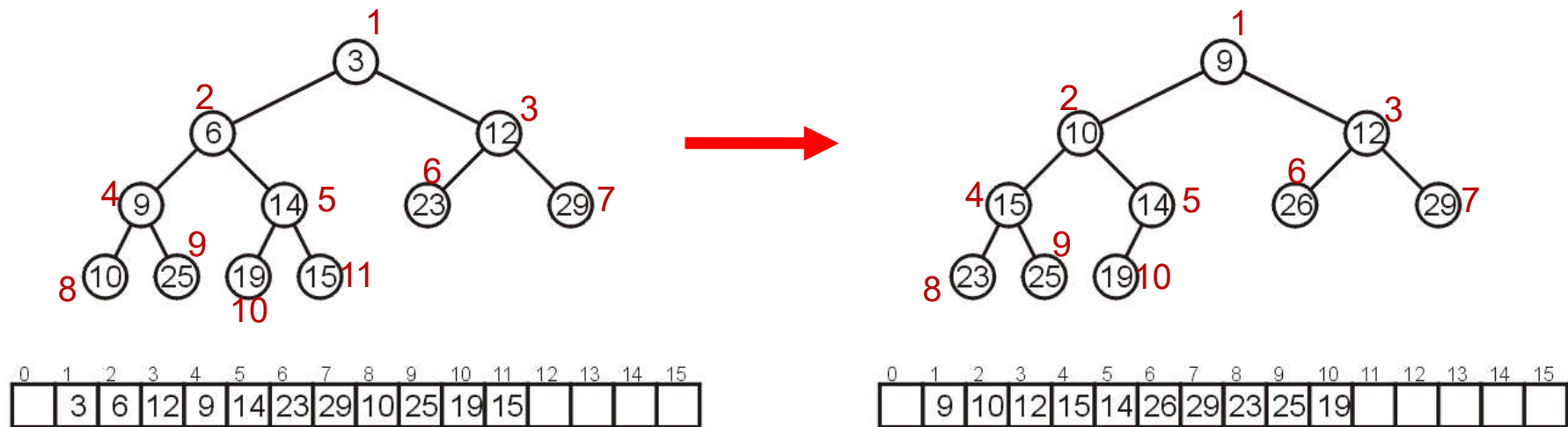
Array Implementation: Pop

The result is a properly formed binary min-heap



Array Implementation: Pop

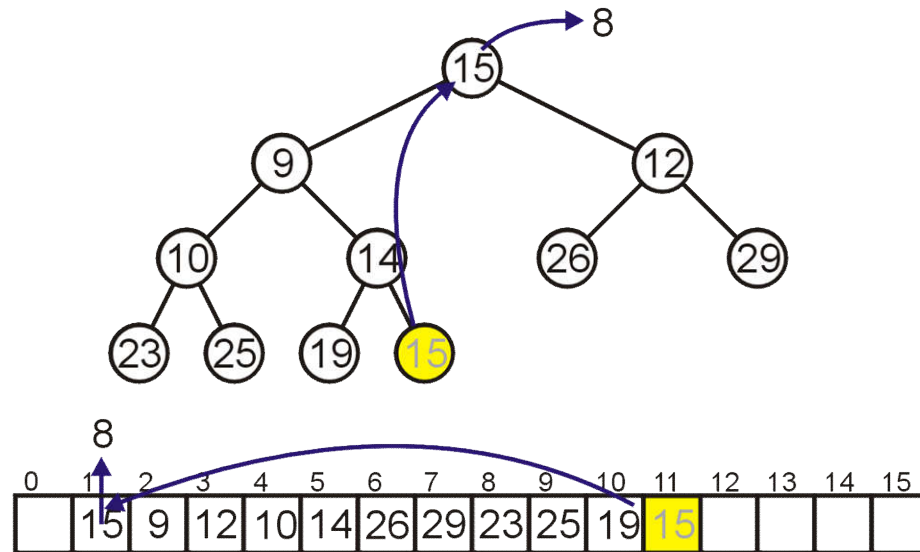
After all our modifications, the final heap is



Array Implementation: Pop

Dequeuing the minimum a third time: **Look at this example again**

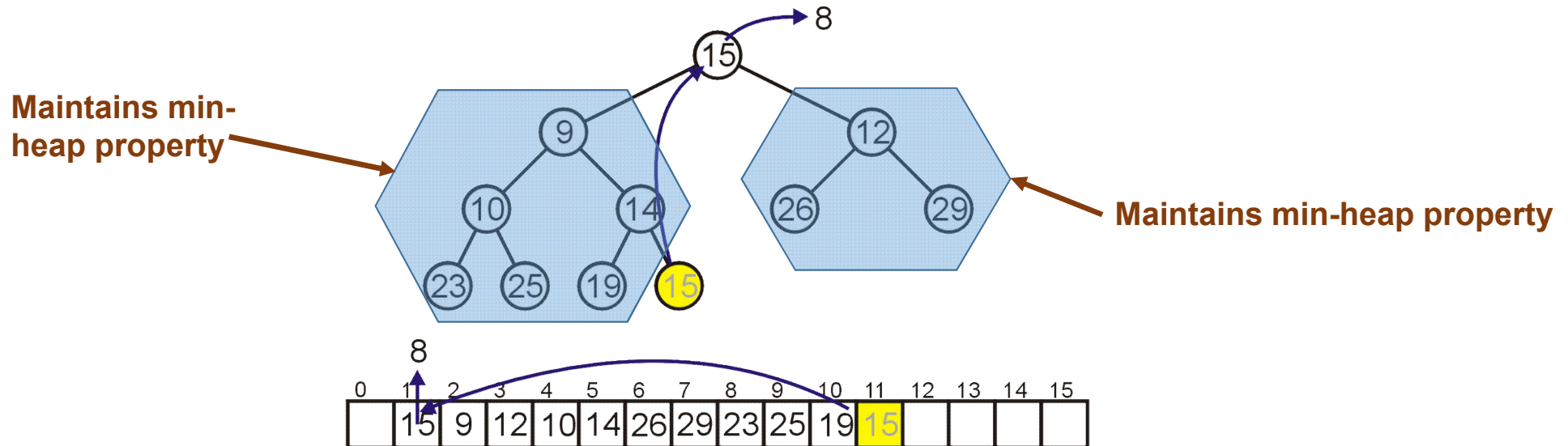
- Copy 15 to the root



Array Implementation: Pop

Dequeuing the minimum a third time: **Look at this example again**

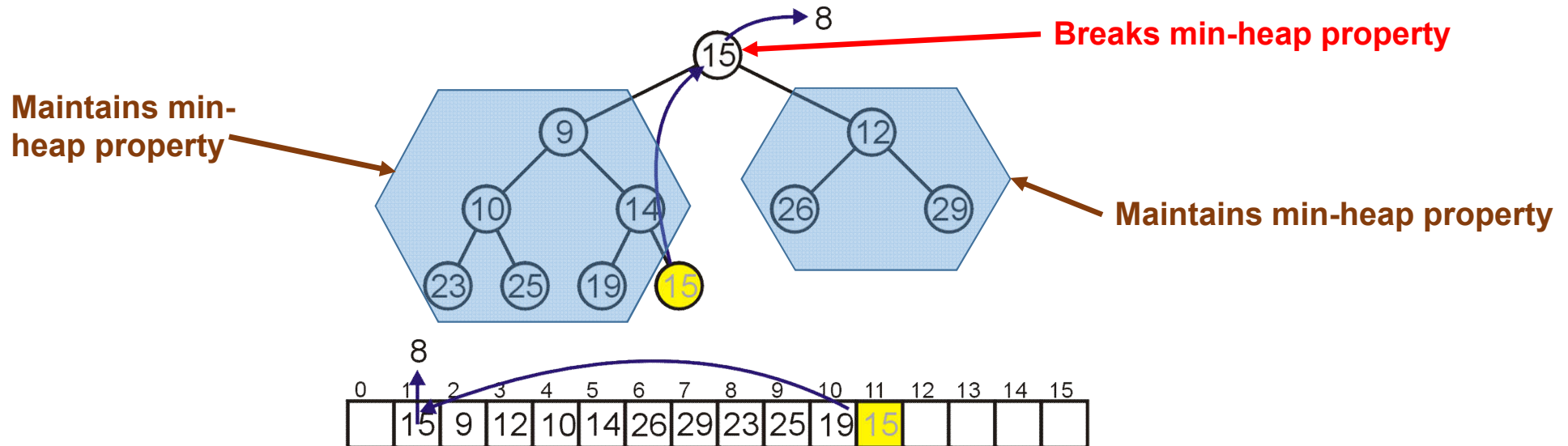
- Copy 15 to the root



Array Implementation: Pop

Dequeuing the minimum a third time: **Look at this example again**

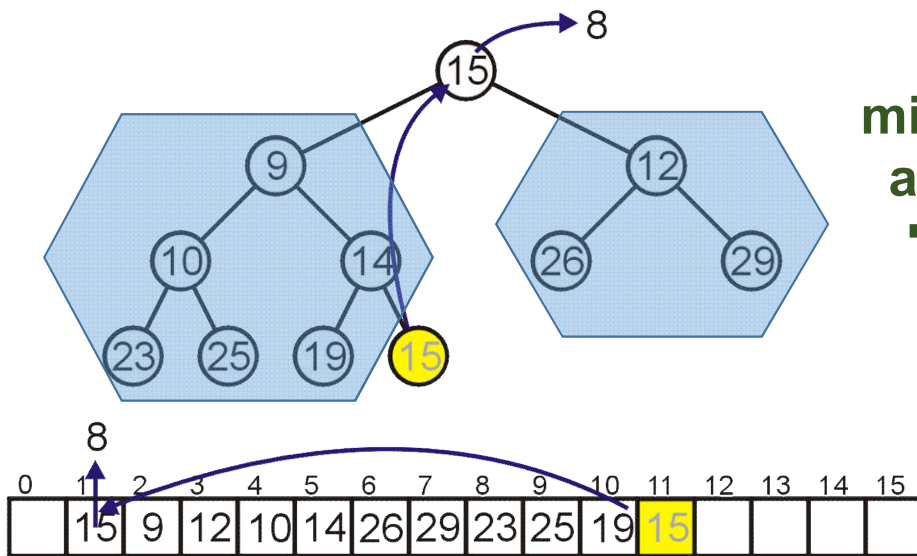
- Copy 15 to the root



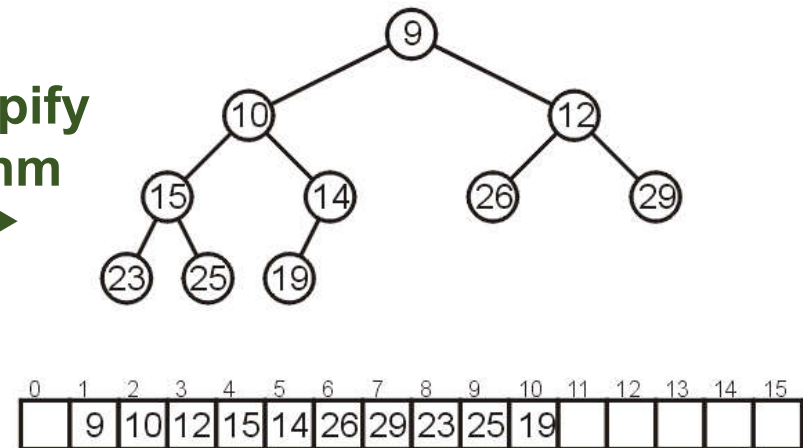
Array Implementation: Pop

Dequeuing the minimum a third time: **Look at this example again**

- Copy 15 to the root



min-heapify
algorithm



MIN-HEAPIFY

MIN-HEAPIFY (A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] < A[i]$ 
4       $\text{smallest} = l$ 
5  else  $\text{smallest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] < A[\text{smallest}]$ 
7       $\text{smallest} = r$ 
8  if  $\text{smallest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{smallest}]$ 
10     MIN-HEAPIFY ( $A, \text{smallest}$ )
```

MIN-HEAPIFY

MIN-HEAPIFY (A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] < A[i]$ 
4       $\text{smallest} = l$ 
5  else  $\text{smallest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] < A[\text{smallest}]$ 
7       $\text{smallest} = r$ 
8  if  $\text{smallest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{smallest}]$ 
10     MIN-HEAPIFY ( $A, \text{smallest}$ )
```

LEFT(i)
1 return $2i$

RIGHT(i)
1 return $2i + 1$

- Its inputs are an array A and an index i into the array.

MIN-HEAPIFY

MIN-HEAPIFY (A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] < A[i]$ 
4       $\text{smallest} = l$ 
5  else  $\text{smallest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] < A[\text{smallest}]$ 
7       $\text{smallest} = r$ 
8  if  $\text{smallest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{smallest}]$ 
10     MIN-HEAPIFY ( $A, \text{smallest}$ )
```

LEFT(i)
1 return $2i$

RIGHT(i)
1 return $2i + 1$

- Its inputs are an array A and an index i into the array.
- When it is called, MIN-HEAPIFY **assumes**
 - the binary trees rooted at LEFT(i) and RIGHT(i) are min-heaps,
 - but that $A[i]$ might be larger than its children
 - thus violating the min-heap property.

MIN-HEAPIFY

MIN-HEAPIFY (A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] < A[i]$ 
4       $\text{smallest} = l$ 
5  else  $\text{smallest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] < A[\text{smallest}]$ 
7       $\text{smallest} = r$ 
8  if  $\text{smallest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{smallest}]$ 
10     MIN-HEAPIFY ( $A, \text{smallest}$ )
```

LEFT(i)
1 return $2i$

RIGHT(i)
1 return $2i + 1$

- Its inputs are an array A and an index i into the array.
- When it is called, MIN-HEAPIFY **assumes**
 - the binary trees rooted at LEFT(i) and RIGHT(i) are min-heaps,
 - but that $A[i]$ might be larger than its children
 - thus violating the min-heap property.
- MIN-HEAPIFY lets the value at $A[i]$ “float down” in the min-heap so that the subtree rooted at index i obeys the min-heap property.

MIN-HEAPIFY

MIN-HEAPIFY (A, i)

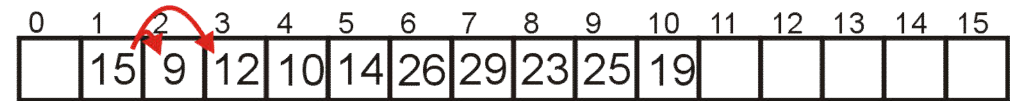
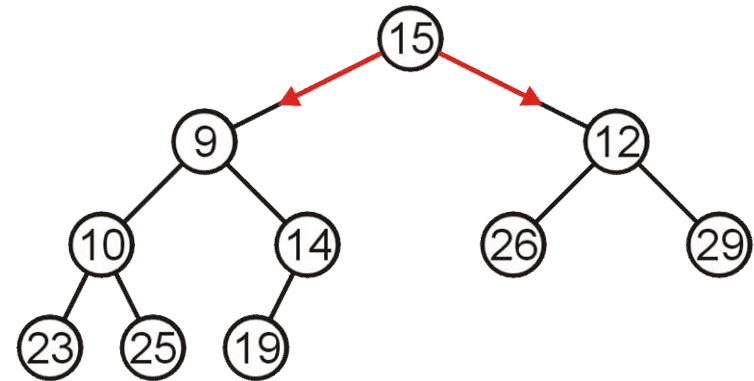
```

1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] < A[i]$ 
4       $\text{smallest} = l$ 
5  else  $\text{smallest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] < A[\text{smallest}]$ 
7       $\text{smallest} = r$ 
8  if  $\text{smallest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{smallest}]$ 
10     MIN-HEAPIFY ( $A, \text{smallest}$ )
    
```

LEFT(i)
1 return $2i$

RIGHT(i)
1 return $2i + 1$

Call MIN-HEAPIFY ($A, 1$)



MIN-HEAPIFY

MIN-HEAPIFY (A, i)

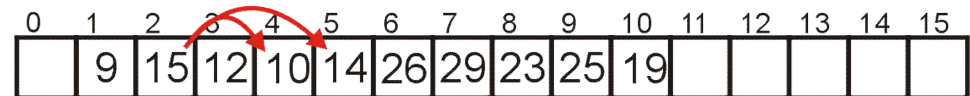
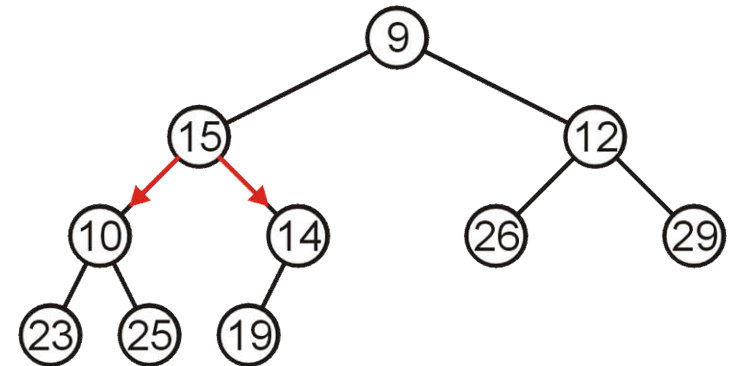
```

1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] < A[i]$ 
4       $\text{smallest} = l$ 
5  else  $\text{smallest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] < A[\text{smallest}]$ 
7       $\text{smallest} = r$ 
8  if  $\text{smallest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{smallest}]$ 
10     MIN-HEAPIFY ( $A, \text{smallest}$ )
    
```

LEFT(i)
1 return $2i$

RIGHT(i)
1 return $2i + 1$

Recursive Call
MIN-HEAPIFY ($A, 2$)



MIN-HEAPIFY

MIN-HEAPIFY (A, i)

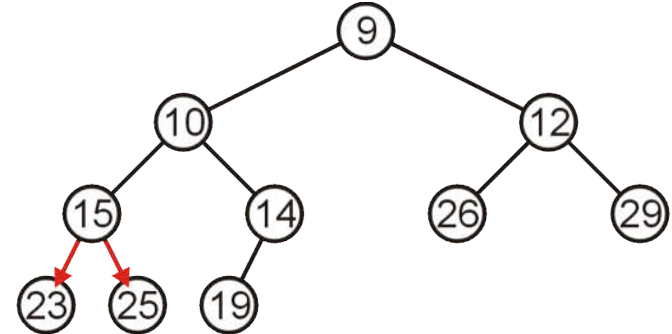
```

1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] < A[i]$ 
4       $\text{smallest} = l$ 
5  else  $\text{smallest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] < A[\text{smallest}]$ 
7       $\text{smallest} = r$ 
8  if  $\text{smallest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{smallest}]$ 
10     MIN-HEAPIFY ( $A, \text{smallest}$ )
    
```

LEFT(i)
1 return $2i$

RIGHT(i)
1 return $2i + 1$

Recursive Call
MIN-HEAPIFY ($A, 4$)



MIN-HEAPIFY

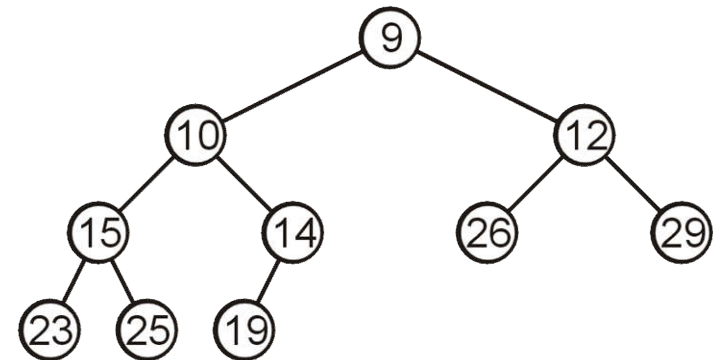
MIN-HEAPIFY (A, i)

```

1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] < A[i]$ 
4       $\text{smallest} = l$ 
5  else  $\text{smallest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] < A[\text{smallest}]$ 
7       $\text{smallest} = r$ 
8  if  $\text{smallest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{smallest}]$ 
10     MIN-HEAPIFY ( $A, \text{smallest}$ )
    
```

LEFT(i)
1 return $2i$

RIGHT(i)
1 return $2i + 1$



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	9	10	12	15	14	26	29	23	25	19					

MAX-HEAPIFY

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```



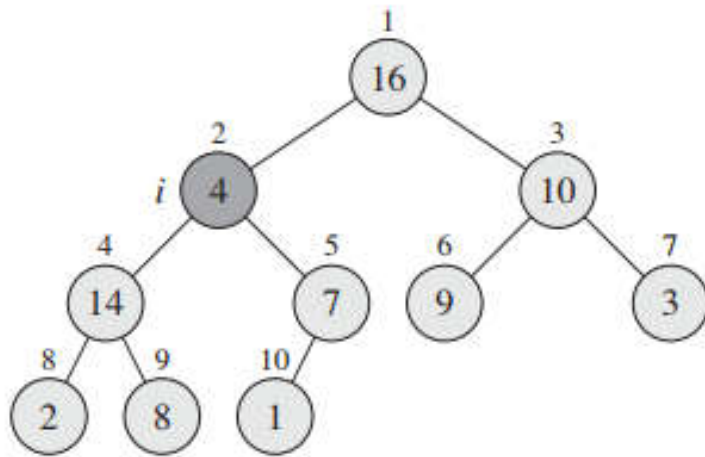
```
LEFT( $i$ )
1  return  $2i$ 

RIGHT( $i$ )
1  return  $2i + 1$ 
```

- Its inputs are an array A and an index i into the array.
- When it is called, **MAX-HEAPIFY** assumes
 - the binary trees rooted at $\text{LEFT}(i)$ and $\text{RIGHT}(i)$ are **max-heaps**,
 - but that $A[i]$ might be **smaller** than its children
 - thus violating the max-heap property.
- **MAX-HEAPIFY** lets the value at $A[i]$ “float down” in the **max-heap** so that the subtree rooted at index i obeys the **max-heap property**.

MAX-HEAPIFY(A,2)

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

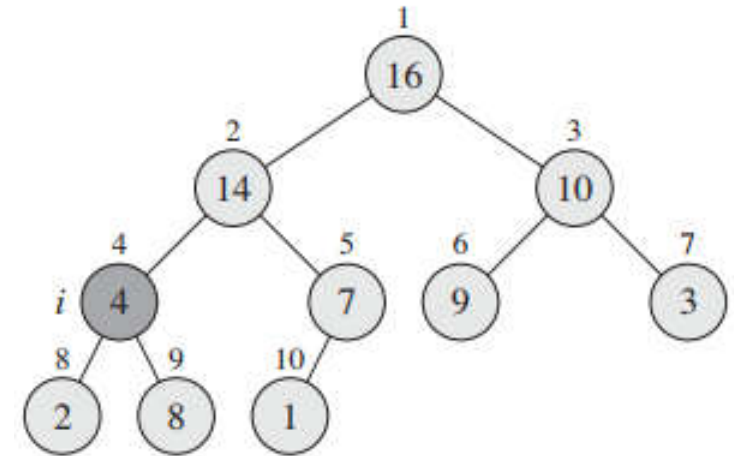
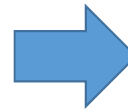
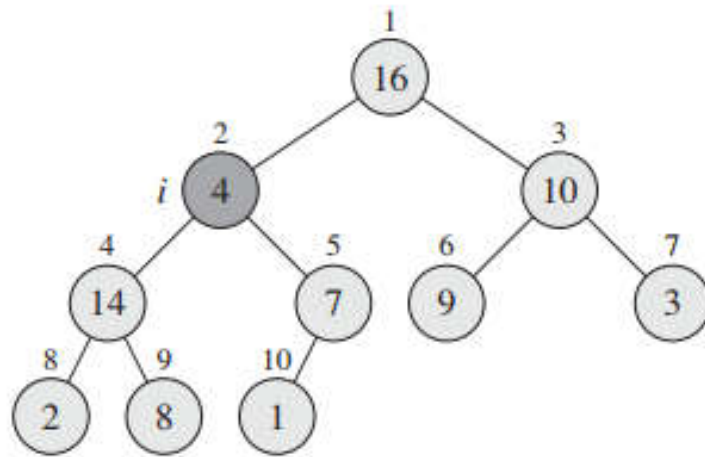


MAX-HEAPIFY(A, i)

```
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4      largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7      largest = r
8  if largest ≠ i
9      exchange A[i] with A[largest]
10     MAX-HEAPIFY(A, largest)
```

MAX-HEAPIFY(A,2)

1	2	3	4	5	6	7	8	9	10
16	14	10	4	7	9	3	2	8	1



MAX-HEAPIFY(A, i)

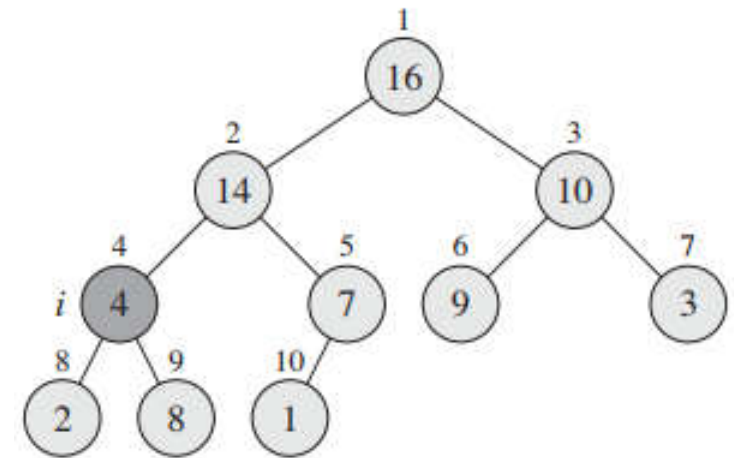
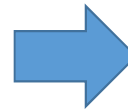
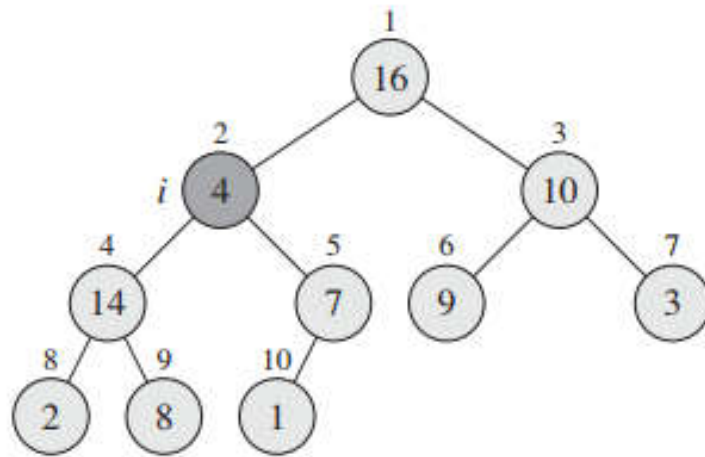
```

1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4      largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7      largest = r
8  if largest ≠ i
9      exchange A[i] with A[largest]
10     MAX-HEAPIFY(A, largest)

```

MAX-HEAPIFY(*A*, 4)

1	2	3	4	5	6	7	8	9	10
16	14	10	4	7	9	3	2	8	1



MAX-HEAPIFY(*A*, *i*)

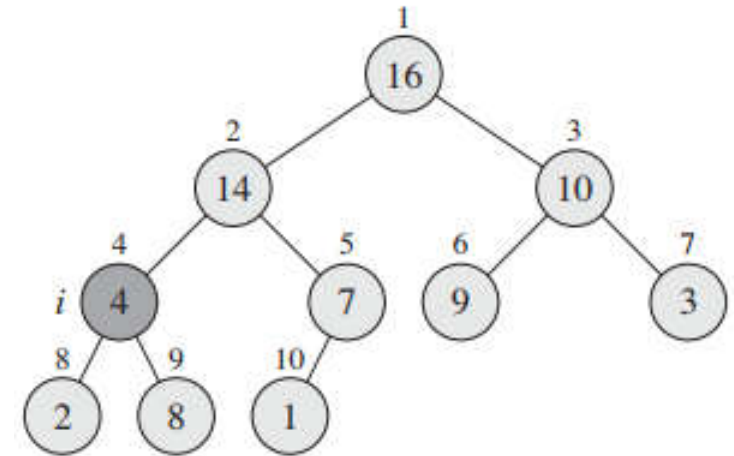
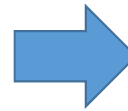
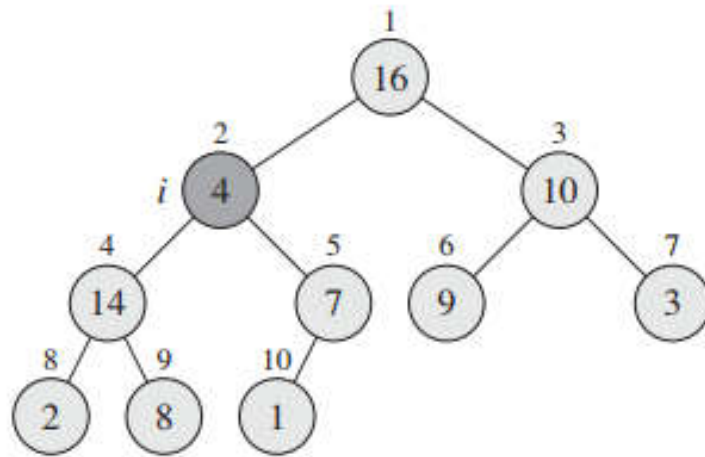
```

1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4      largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7      largest = r
8  if largest ≠ i
9      exchange A[i] with A[largest]
10     MAX-HEAPIFY(A, largest)

```

MAX-HEAPIFY(A, 4)

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

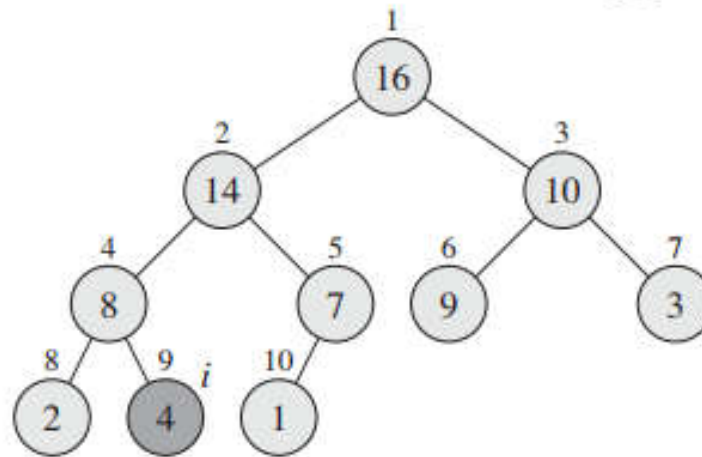


MAX-HEAPIFY(A, i)

```

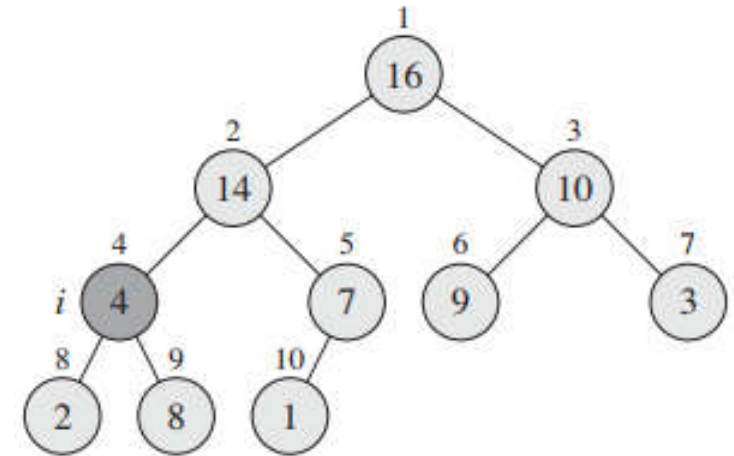
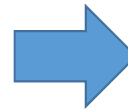
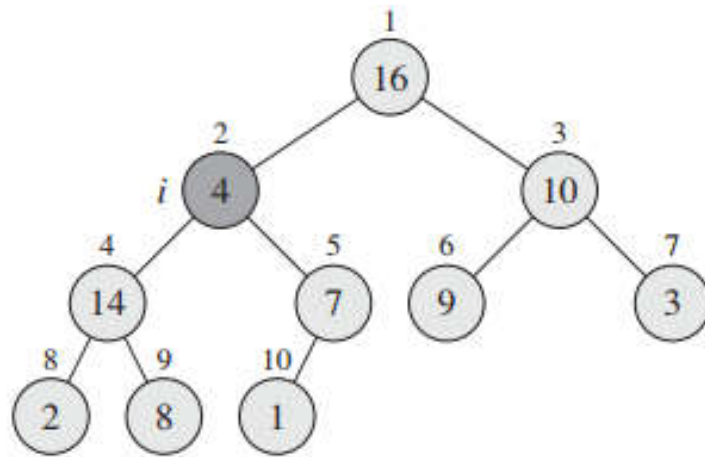
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4      largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7      largest = r
8  if largest ≠ i
9      exchange A[i] with A[largest]
10     MAX-HEAPIFY(A, largest)

```



MAX-HEAPIFY(A, 9)

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

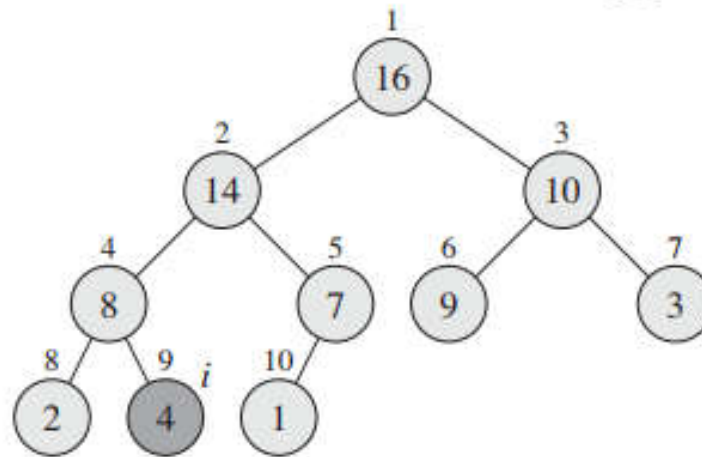


MAX-HEAPIFY(A, i)

```

1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4      largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7      largest = r
8  if largest ≠ i
9      exchange A[i] with A[largest]
10     MAX-HEAPIFY(A, largest)

```

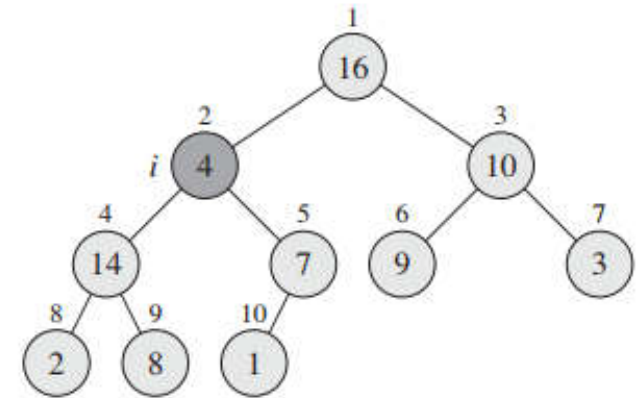


(MAX/MIN)-HEAPIFY: Running time

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

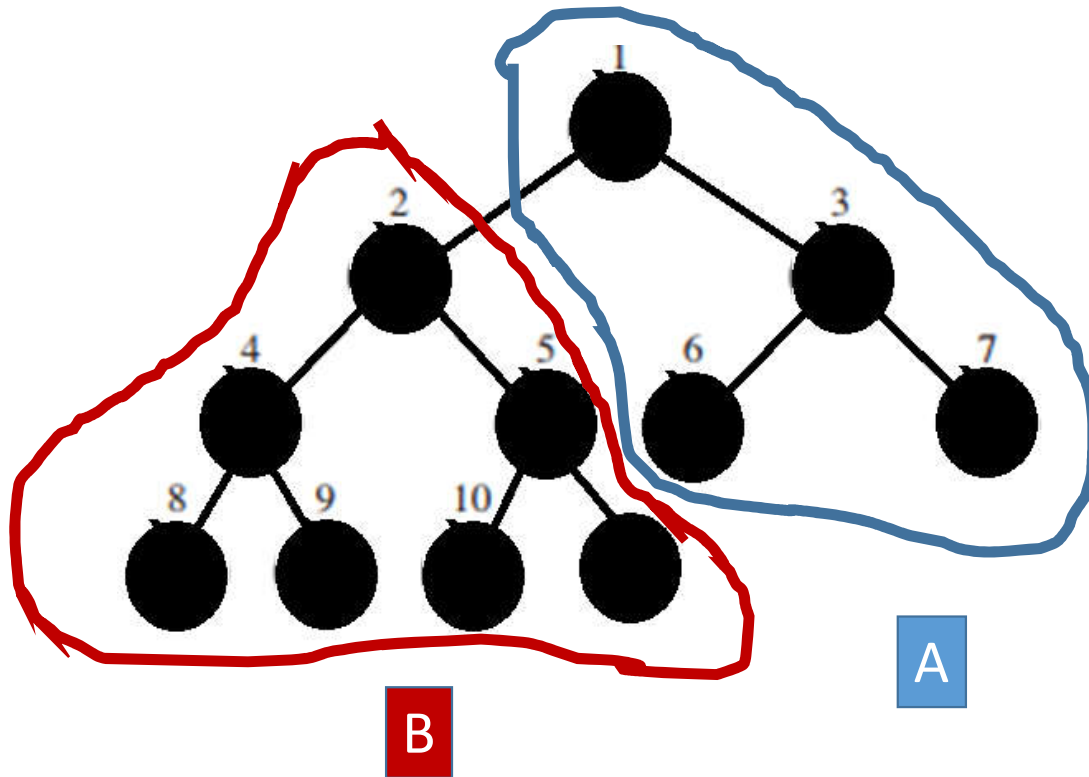
$\Theta(1)$



What is the worst case scenario?

If MAX-HEAPIFY is called on an array/tree of n nodes how many nodes are there in the sub-tree/array on which the recursive call is made?





- The children's subtrees each have size at most $2n/3$
 - $A + B = n$
 $\Rightarrow B \approx 2A$
 $\Rightarrow A \approx n/3$
 $\Rightarrow B \approx 2n/3$
- The worst case occurs when the bottom level of the tree is exactly half full

(MAX/MIN)-HEAPIFY: Running time

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```



$\Theta(1)$

$$T(n) \leq T\left(\frac{2n}{3}\right) + \Theta(1)$$
$$\Rightarrow T(n) = O(\log n)$$

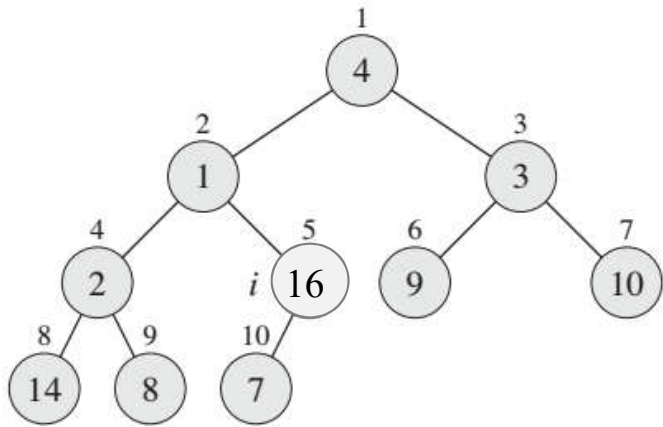
Master Theorem

BUILD-(MAX/MIN)-HEAP

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

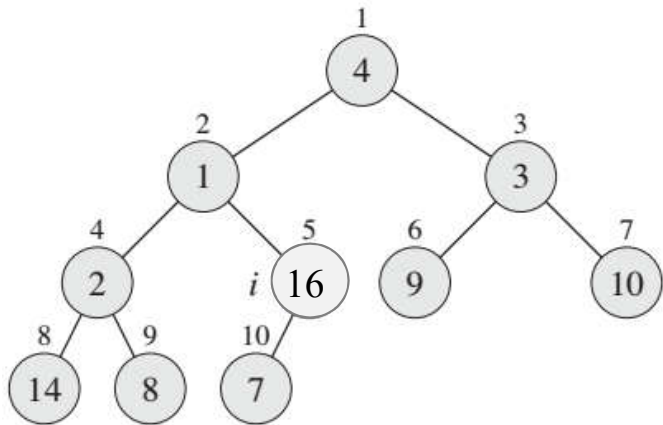
BUILD-MAXHEAP

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7



BUILD-MAXHEAP

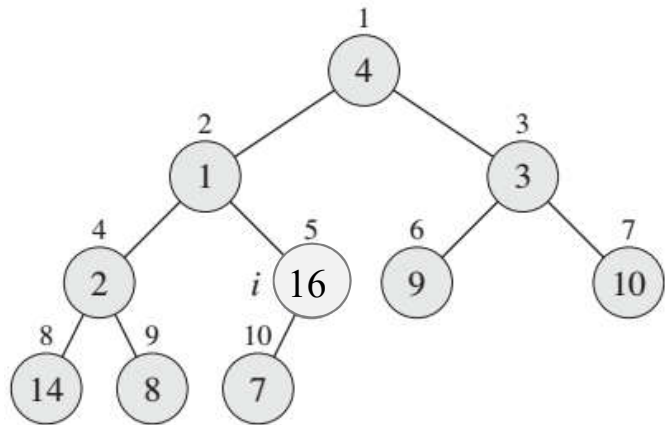
1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7



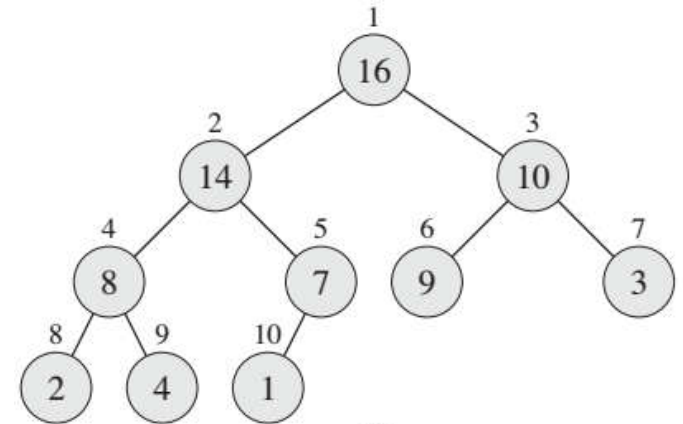
- We can use the procedure MAX-HEAPIFY in a **bottom-up** manner to convert an array into a max-heap.

BUILD-MAXHEAP

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7



- We can use the procedure MAX-HEAPIFY in a **bottom-up** manner to convert an array into a max-heap.



1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

BUILD-MAXHEAP

BUILD-MAX-HEAP(A)

```
1 A.heap-size = A.length  
2 for  $i = \lfloor A.length/2 \rfloor$  downto 1  
3     MAX-HEAPIFY(A, i)
```

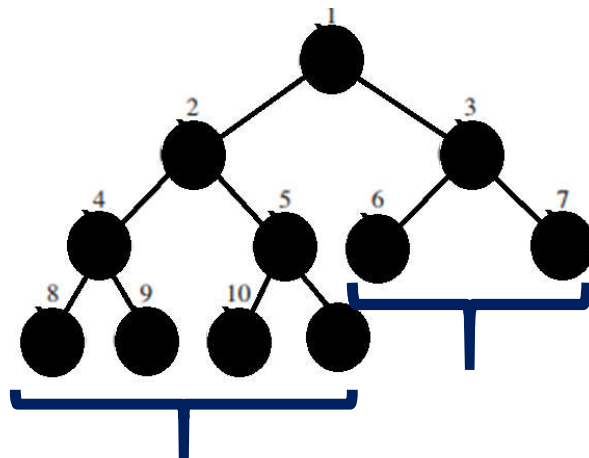
- We can use the procedure MAX-HEAPIFY in a **bottom-up** manner to convert an array into a max-heap.

BUILD-(MAX/MIN)-HEAP

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)

$n = 11$
 $n/2 + 1 = 6$
 $\therefore$ 6 onward all are leaves



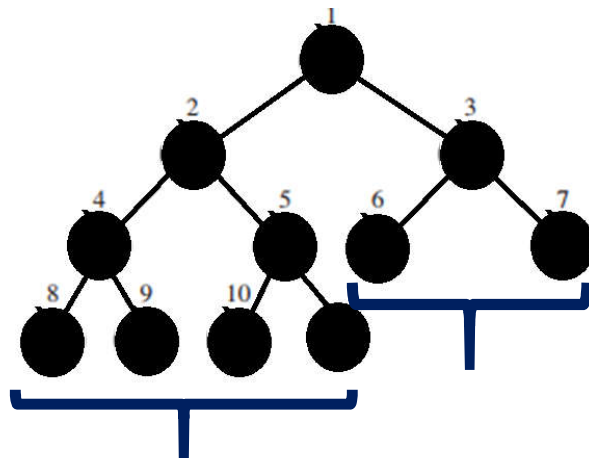
- We can use the procedure MAX-HEAPIFY in a bottom-up manner to convert an array into a max-heap.
- The elements in the subarray $A[\lfloor n/2 \rfloor + 1], \dots, A[n]$ are all leaves.
 - Each is a 1-element heap.

BUILD-(MAX/MIN)-HEAP

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)

$n = 11$
 $n/2 + 1 = 6$
 $\therefore$ 6 onward all are leaves



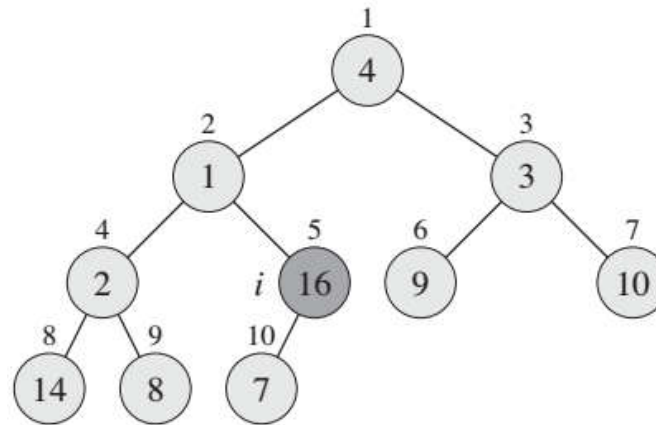
- We can use the procedure MAX-HEAPIFY in a bottom-up manner to convert an array into a max-heap.
- The elements in the subarray $A[\lfloor n/2 \rfloor + 1], \dots, A[n]$ are all leaves.
 - Each is a 1-element heap.
- BUILD-MAX-HEAP goes through the remaining nodes *upward* and call MAX-HEAPIFY on each one.

BUILD-MAX-HEAP

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

BUILD-MAX-HEAP(A)

- 1 $A.\text{heap-size} = A.\text{length}$
- 2 **for** $i = \lfloor A.\text{length}/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)



BUILD-MAX-HEAP

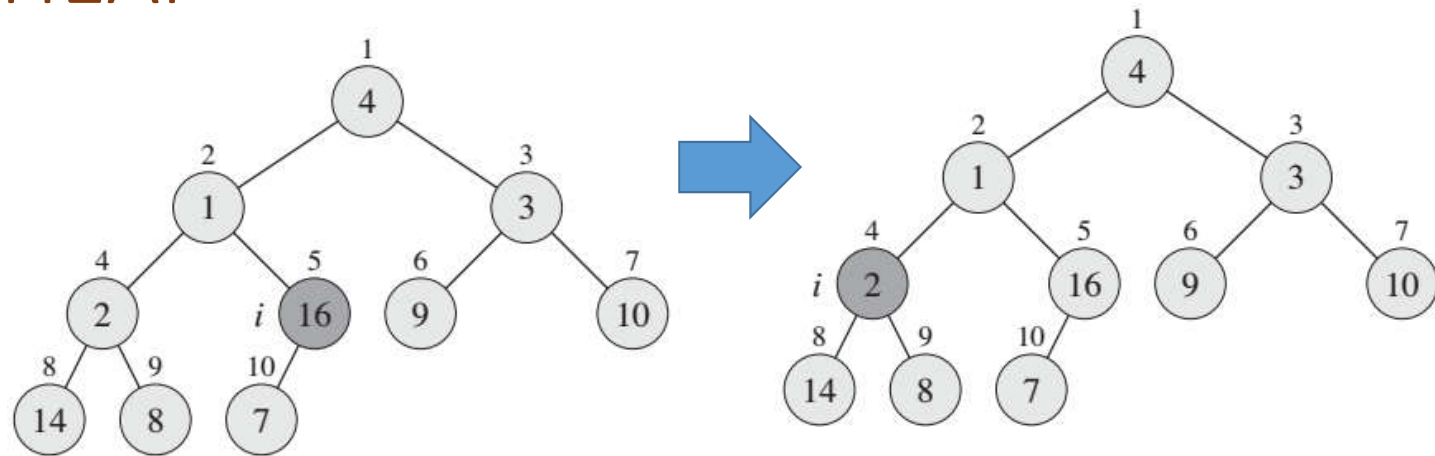
1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

BUILD-MAX-HEAP(A)

1 $A.heap-size = A.length$

2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1

3 MAX-HEAPIFY(A, i)

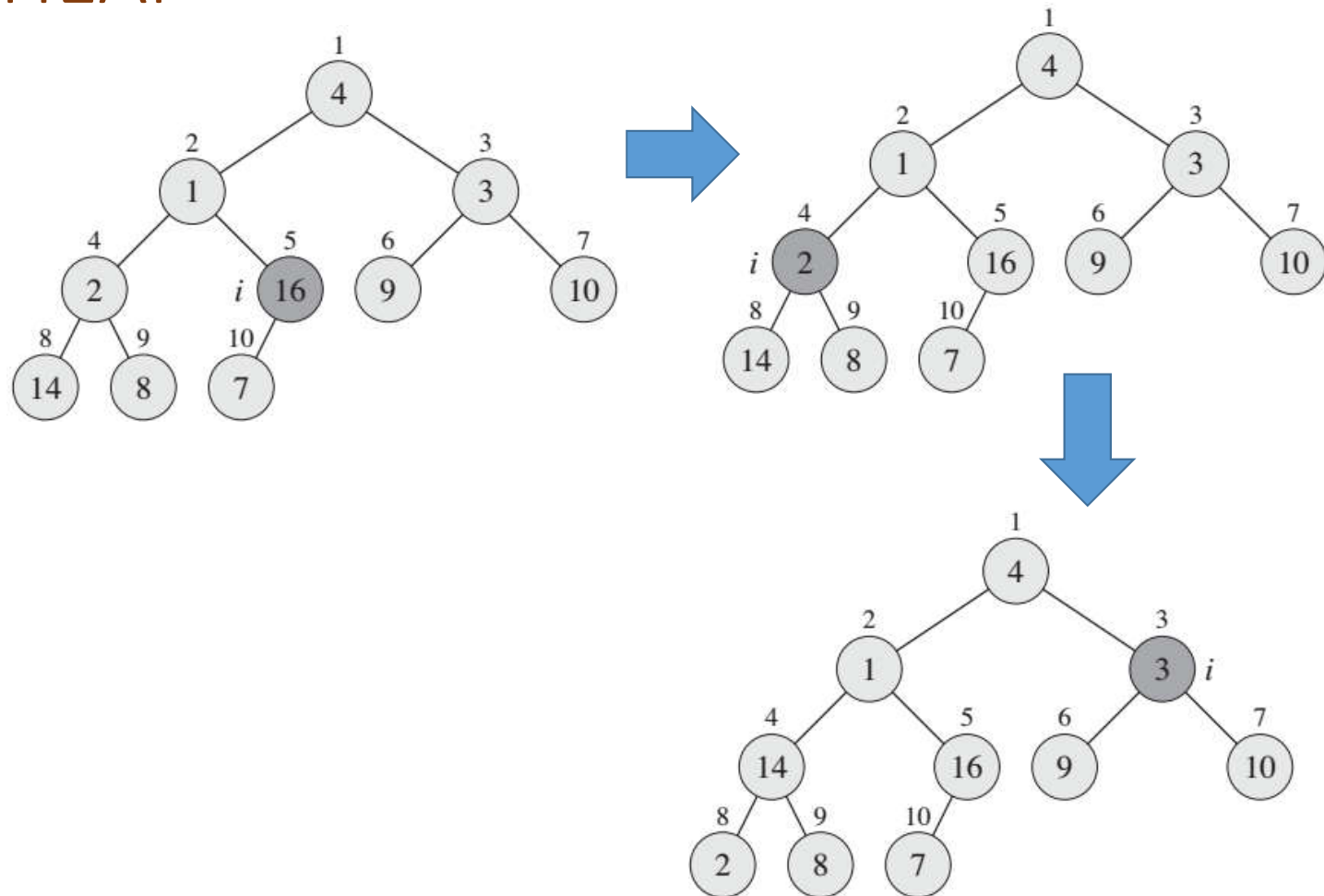


BUILD-MAX-HEAP

1	2	3	4	5	6	7	8	9	10
4	1	3	14	16	9	10	2	8	7

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)



BUILD-MAX-HEAP

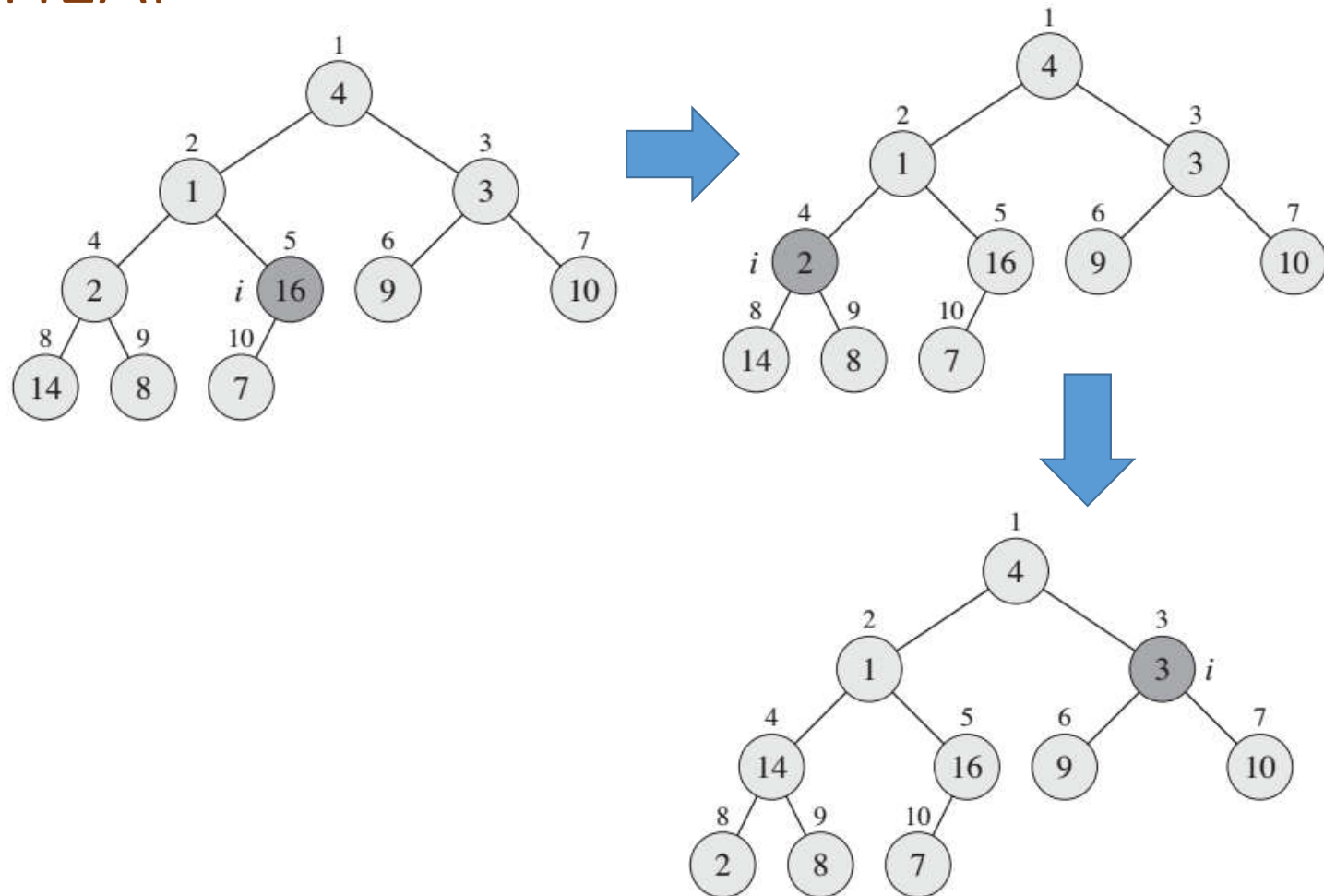
1	2	3	4	5	6	7	8	9	10
4	1	3	14	16	9	10	2	8	7

BUILD-MAX-HEAP(A)

1 $A.heap\text{-}size = A.length$

2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1

3 MAX-HEAPIFY(A, i)

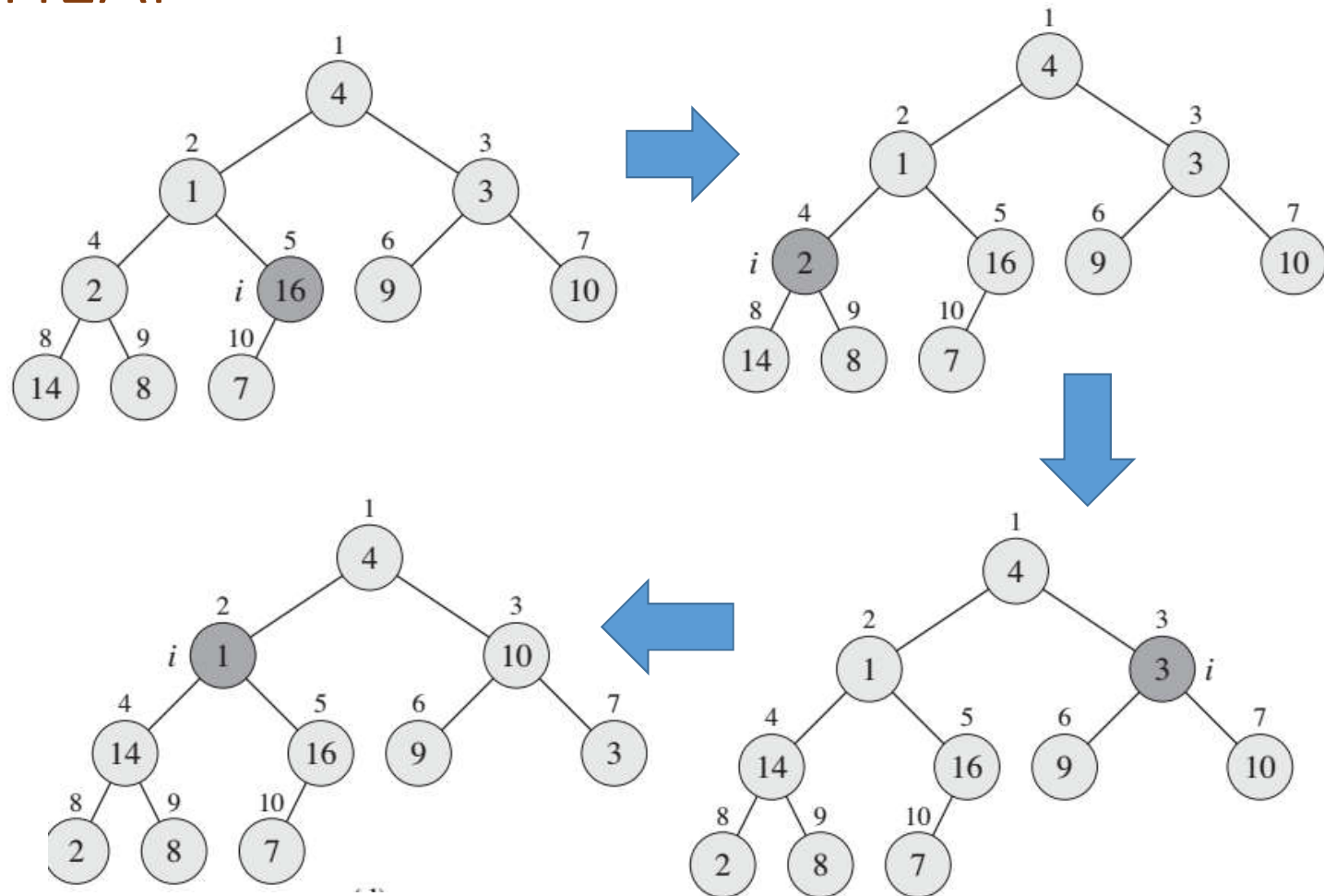


BUILD-MAX-HEAP

1	2	3	4	5	6	7	8	9	10
4	1	10	14	16	9	3	2	8	7

BUILD-MAX-HEAP(A)

- 1 $A.heap-size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)

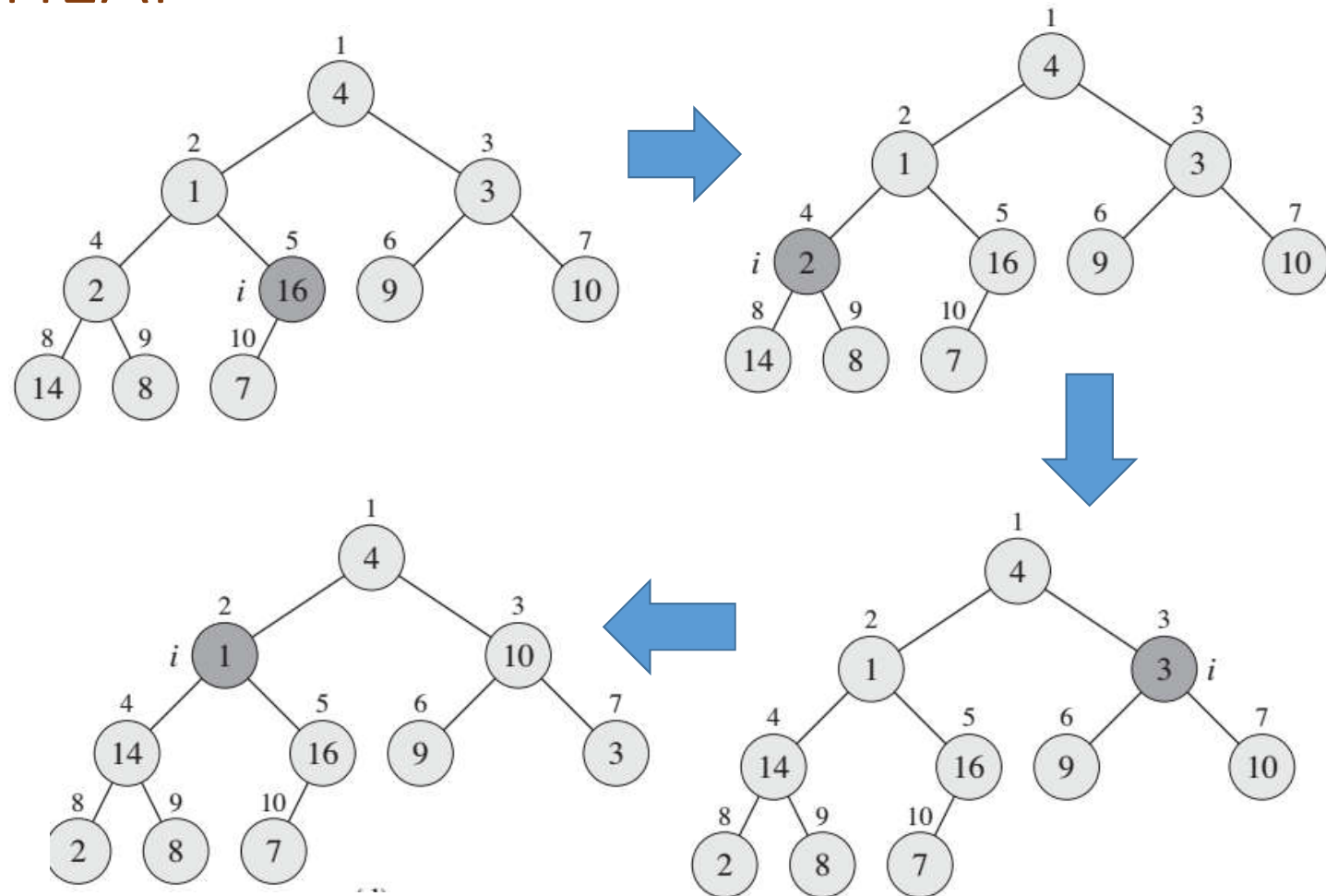


BUILD-MAX-HEAP

1	2	3	4	5	6	7	8	9	10
4	1	10	14	16	9	3	2	8	7

BUILD-MAX-HEAP(A)

- 1 $A.heap-size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)

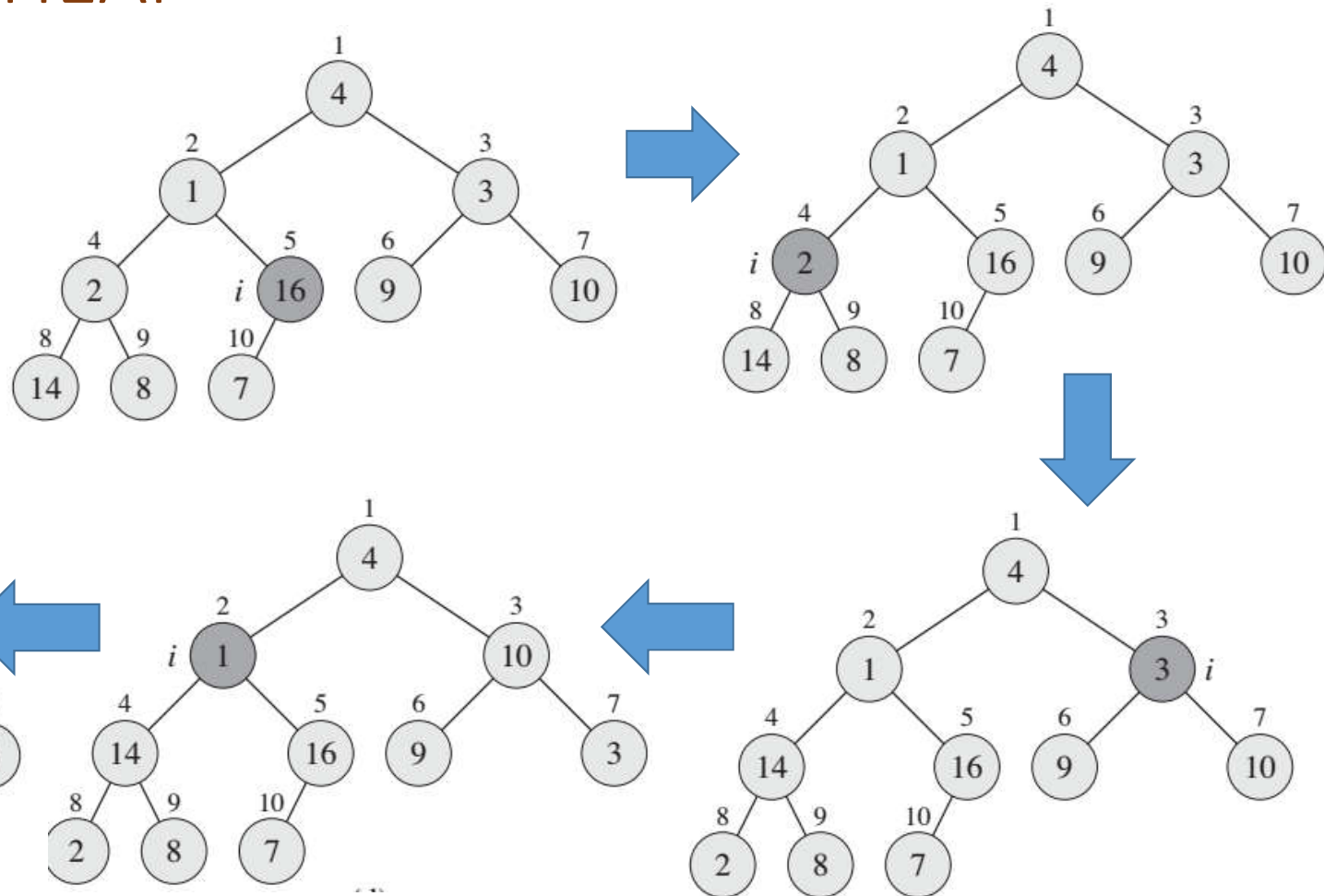


BUILD-MAX-HEAP

1	2	3	4	5	6	7	8	9	10
4	16	10	14	7	9	3	2	8	1

BUILD-MAX-HEAP(A)

- 1 $A.\text{heap-size} = A.\text{length}$
- 2 **for** $i = \lfloor A.\text{length}/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)

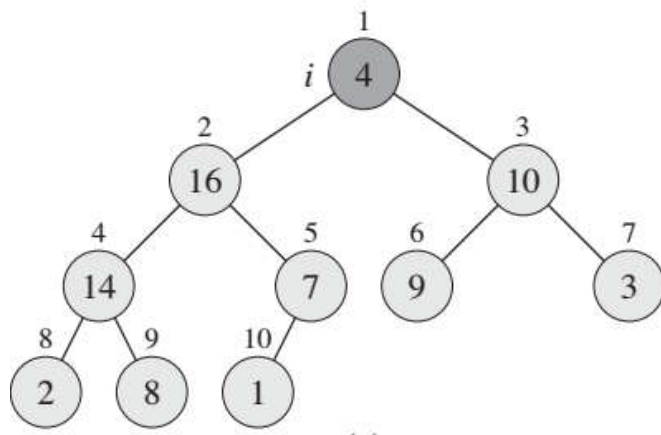


BUILD-MAX-HEAP

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)

1	2	3	4	5	6	7	8	9	10
4	16	10	14	7	9	3	2	8	1

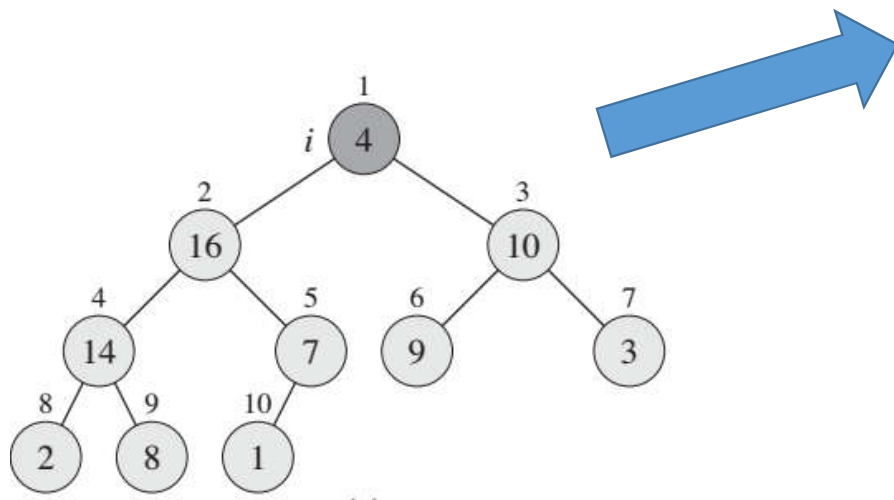
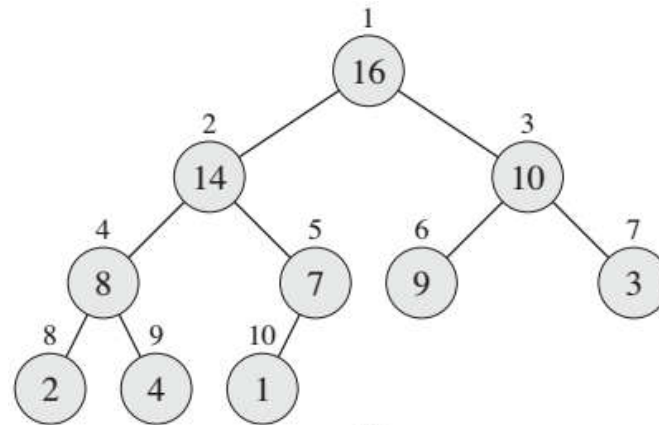


BUILD-MAX-HEAP

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



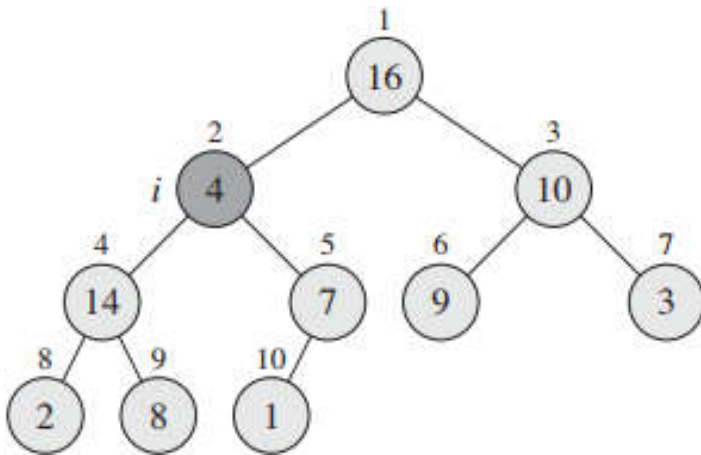
BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

1 $A.heap\text{-}size = A.length$

2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1

3 MAX-HEAPIFY(A, i)

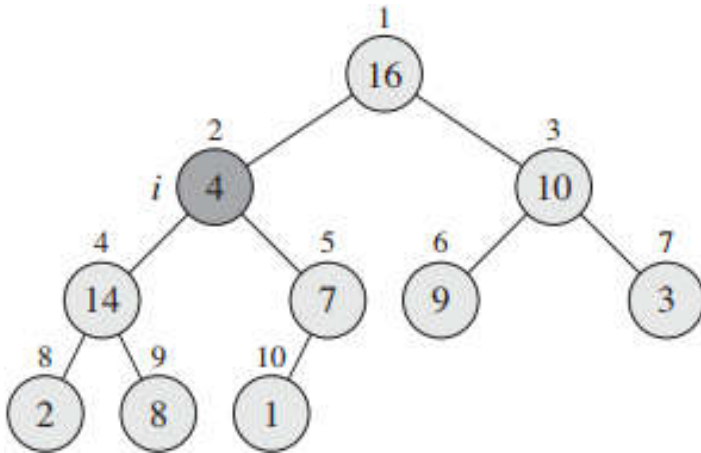


1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

```
1 A.heap-size = A.length
2 for i =  $\lfloor A.length/2 \rfloor$  downto 1
3   MAX-HEAPIFY(A, i)
```



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

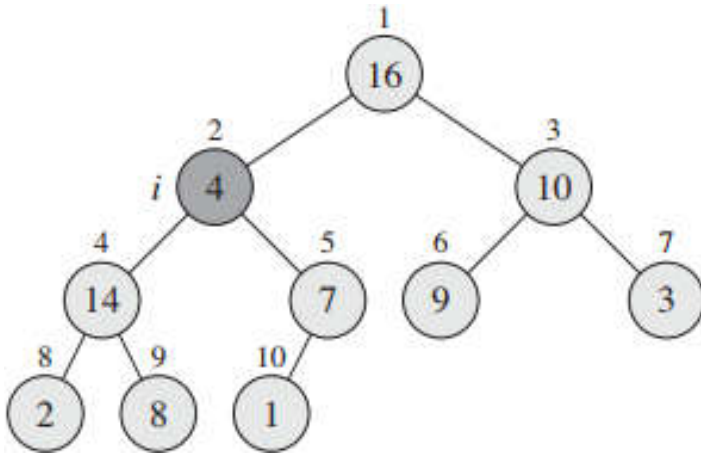
Observe the following loop invariant

At the start of each iteration of the **for** loop of lines 2-3,
each node $i+1, i+2, \dots, n$ is the root of a max-heap

BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

Observe the following loop invariant

At the start of each iteration of the **for** loop of lines 2-3, each node $i+1, i+2, \dots, n$ is the root of a max-heap

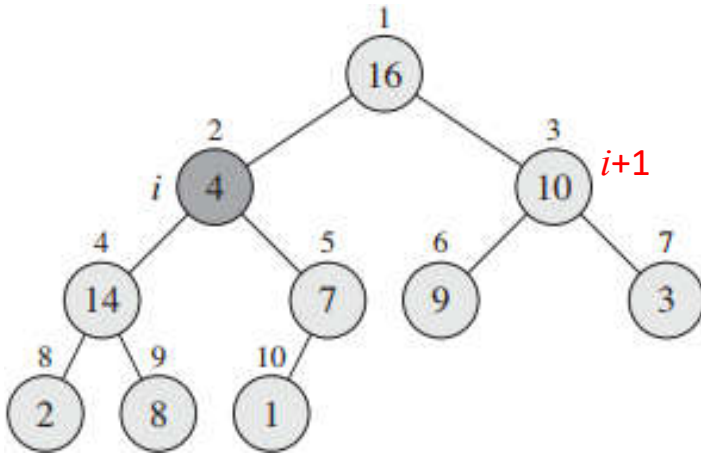
At Initialization

$i = \lfloor n/2 \rfloor$: all nodes afterwards, e.g., $\lfloor n/2 \rfloor + 1, \dots, n$ are leaf-nodes, therefore, root of max-heaps.

BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

Observe the following loop invariant

At the start of each iteration of the **for** loop of lines 2-3,
each node $i+1, i+2, \dots, n$ is the root of a max-heap

At Maintenance Steps

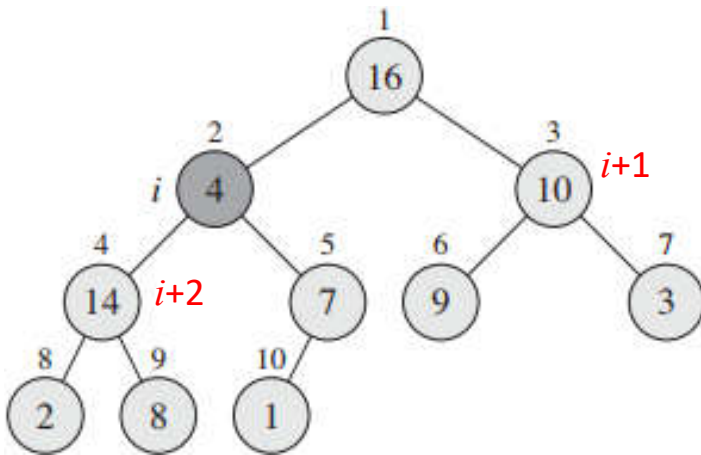
Let, it's true for $i + 1$.

We have to show that it is true for i .

BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

Observe the following loop invariant

At the start of each iteration of the **for** loop of lines 2-3,
each node $i+1, i+2, \dots, n$ is the root of a max-heap

At Maintenance Steps

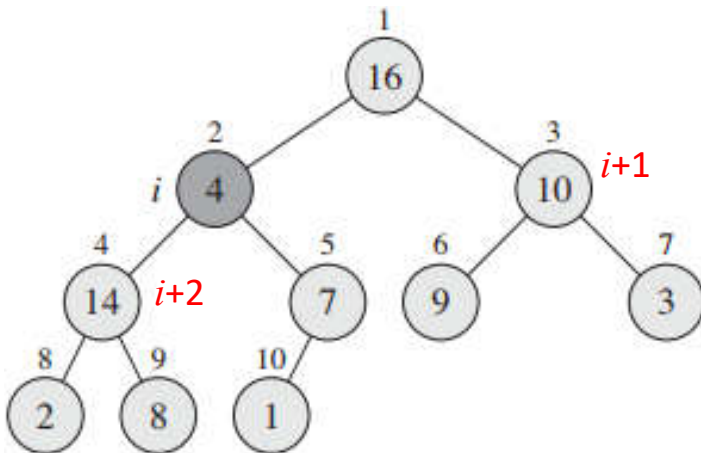
Let, it's true for $i + 1$.

All nodes from $i + 2$ to n are roots of max-heaps.

BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

Observe the following loop invariant

At the start of each iteration of the **for** loop of lines 2-3,
each node $i+1, i+2, \dots, n$ is the root of a max-heap

At Maintenance Steps

Let, it's true for $i + 1$.

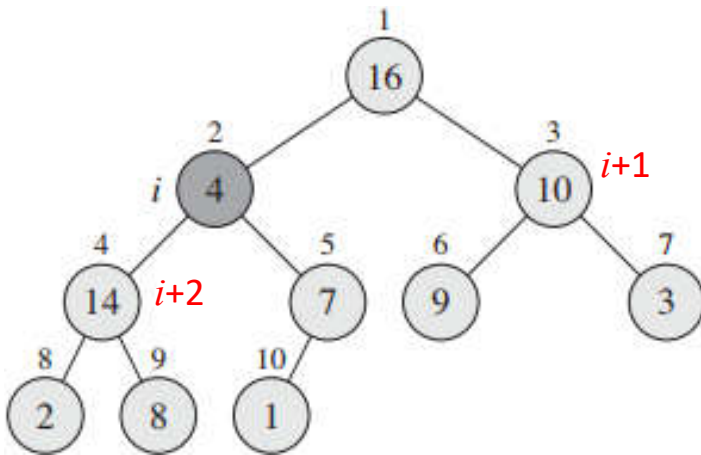
All nodes from $i + 2$ to n are roots of max-heaps.

Children of node $i+1$ are **higher than** $i+1$
e.g., They are in $i + 2$ to n

BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

- 1 $A.heap\text{-}size = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
- 3 MAX-HEAPIFY(A, i)



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

Observe the following loop invariant

At the start of each iteration of the **for** loop of lines 2-3, each node $i+1, i+2, \dots, n$ is the root of a max-heap

At Maintenance Steps

Let, it's true for $i + 1$.

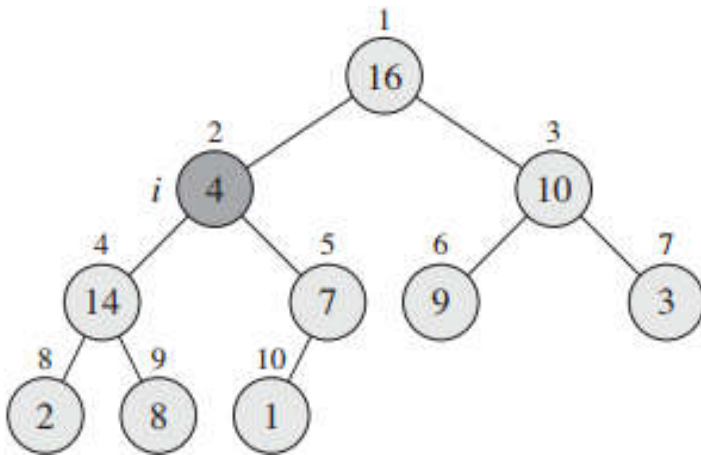
All nodes from $i + 2$ to n are roots of max-heaps.

Therefore, max-heapify will correctly make node $i+1$ a root of a max-heap

BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

```
1 A.heap-size = A.length
2 for i =  $\lfloor A.length/2 \rfloor$  downto 1
3   MAX-HEAPIFY(A, i)
```



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

Observe the following loop invariant

At the start of each iteration of the **for** loop of lines 2-3, each node $i+1, i+2, \dots, n$ is the root of a max-heap

At Maintenance Steps

Let, it's true for $i + 1$.

All nodes from $i + 2$ to n are roots of max-heaps.

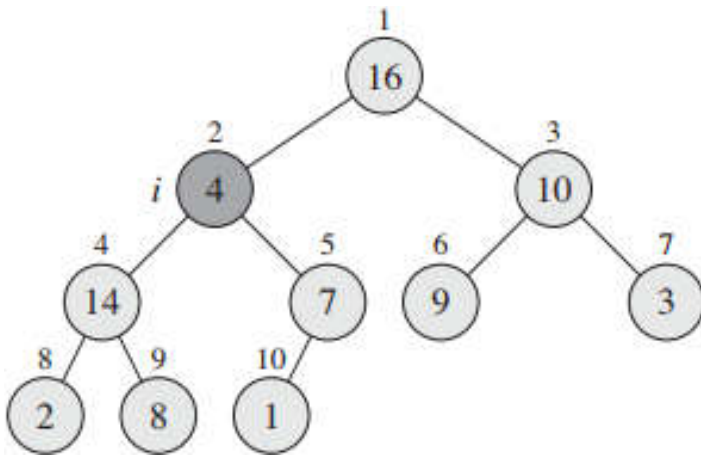
After that iteration, node $i+1$ will be root of a max-heap.

Therefore, all nodes from $i + 1$ to n are roots of max-heaps.

BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

```
1 A.heap-size = A.length
2 for i =  $\lfloor A.length/2 \rfloor$  downto 1
3   MAX-HEAPIFY(A, i)
```



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

Observe the following loop invariant

At the start of each iteration of the **for** loop of lines 2-3, each node $i+1, i+2, \dots, n$ is the root of a max-heap

At Maintenance Steps

Let, it's true for $i + 1$.

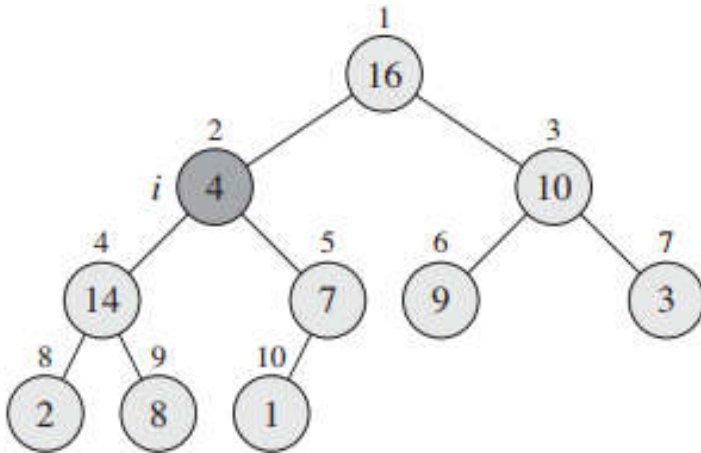
We get: All nodes from $i + 1$ to n are root of max-heaps.

Therefore, at the start of next iteration (e.g., at i) each node $i+1, i+2, \dots, n$ is the root of a max-heap

BUILD-(MAX/MIN)-HEAP: Correctness

BUILD-MAX-HEAP(A)

```
1 A.heap-size = A.length
2 for i =  $\lfloor A.length/2 \rfloor$  downto 1
3   MAX-HEAPIFY(A, i)
```



Observe the following loop invariant

At the start of each iteration of the **for** loop of lines 2-3,
each node $i+1, i+2, \dots, n$ is the root of a max-heap

After Termination

Value of $i = 0$.

All nodes from 1 to n are roots of max-heaps.

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

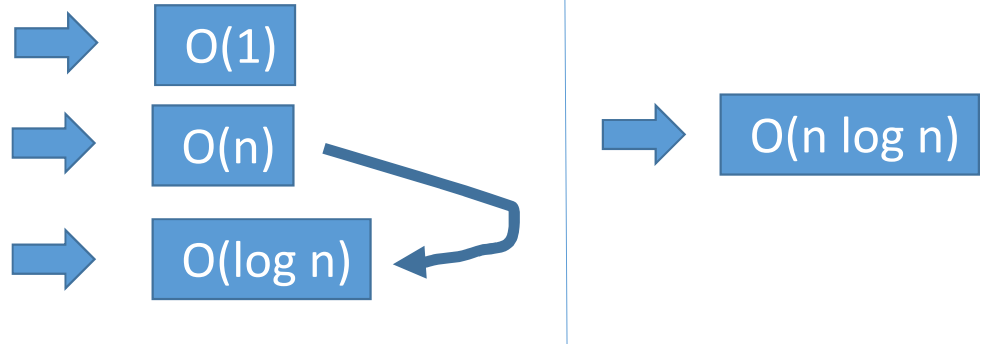
BUILD-(MAX/MIN)-HEAP: Running Time (Simple)

BUILD-MAX-HEAP(A)

1 $A.heap-size = A.length$

2 **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1

3 MAX-HEAPIFY(A, i)

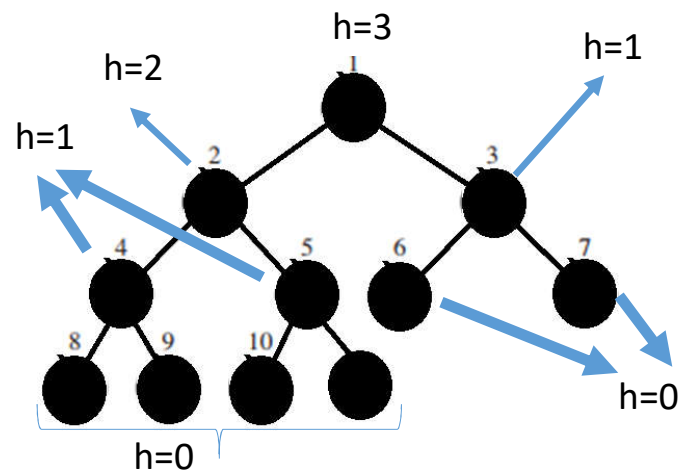


BUILD-(MAX/MIN)-HEAP: Running Time (Tighter)

BUILD-MAX-HEAP(A)

```
1 A.heap-size = A.length
2 for i =  $\lfloor A.length/2 \rfloor$  downto 1
3   MAX-HEAPIFY(A, i)
```

- an n -element heap has **height** $\lfloor \log n \rfloor$
- There are at most $\lceil n/2^{h+1} \rceil$ **nodes** at any **height** h .
- Here height is the longest distance from a leaf
 - Somewhat opposite to depth



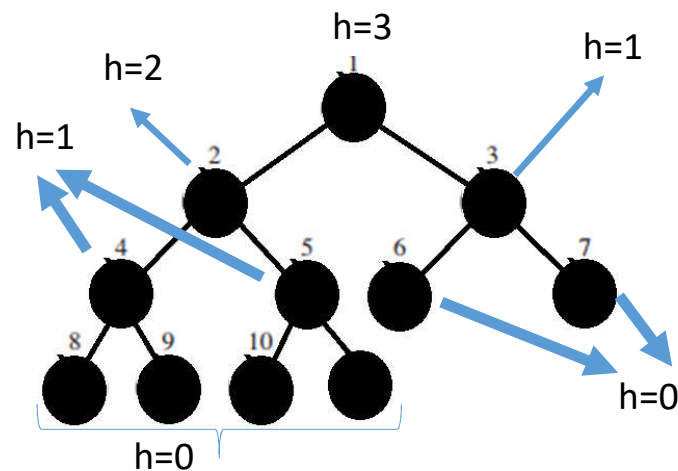
BUILD-(MAX/MIN)-HEAP: Running Time (Tighter)

BUILD-MAX-HEAP(A)

```
1 A.heap-size = A.length
2 for i =  $\lfloor A.length/2 \rfloor$  downto 1
3   MAX-HEAPIFY(A, i)
```

- an n -element heap has **height** $\lfloor \log n \rfloor$
- There are at most $\lceil n/2^{h+1} \rceil$ **nodes** at any **height** h .
- Here height is the longest distance from a leaf
 - Somewhat opposite to depth

Complexity =
$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right)$$



BUILD-(MAX/MIN)-HEAP: Running Time (Tighter)

BUILD-MAX-HEAP(A)

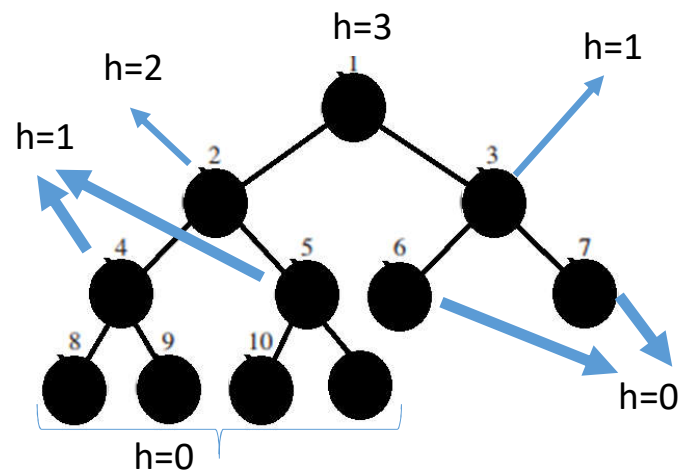
```
1 A.heap-size = A.length
2 for i =  $\lfloor A.length/2 \rfloor$  downto 1
3   MAX-HEAPIFY(A, i)
```

- an n -element heap has **height** $\lfloor \log n \rfloor$
- There are at most $\lceil n/2^{h+1} \rceil$ **nodes** at any **height** h .
- Here height is the longest distance from a leaf
 - Somewhat opposite to depth

$$\text{Complexity} = \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right)$$

We know, asymptotically

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1 - 1/2)^2} = 2$$



BUILD-(MAX/MIN)-HEAP: Running Time (Tighter)

BUILD-MAX-HEAP(A)

```
1 A.heap-size = A.length
2 for i =  $\lfloor A.length/2 \rfloor$  downto 1
3   MAX-HEAPIFY(A, i)
```

- an n -element heap has **height** $\lfloor \log n \rfloor$
- There are at most $\lceil n/2^{h+1} \rceil$ **nodes** at any **height** h .
- Here height is the longest distance from a leaf
 - Somewhat opposite to depth

$$\text{Complexity} = \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right)$$

We know, asymptotically

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1 - 1/2)^2} = 2$$

Therefore,

$$\begin{aligned} \text{Complexity} &= O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) \\ &= O(n) . \end{aligned}$$

